

Sección 9: Arquitectura Orientada a Eventos & Sagas (RabbitMQ)

Arquitectura orientada a eventos & Sagas (RabbitMQ)

Infraestructura y configuración

- Integración de RabbitMQ con Docker Compose
- Configuración de MessageConverter para serialización JSON

Flujo principal de negocio (Happy Path)

- Definición del evento inicial OrderPlacedEvent
- Producer en Order Service
- Consumer en Inventory Service
- Estrategia Fan-Out y diseño del Notification Service
- Integración con Mailtrap para pruebas en desarrollo
- Paso a producción con Gmail/SMTP real y perfiles de Spring

Patrón Saga: validación y compensación

- Contratos de la saga: eventos de confirmación y de fallo
- Inventory como validador de reglas de negocio
- Notificaciones basadas en certezas (sin ruido innecesario)
- Consistencia eventual en Order Service

Robustez técnica y resiliencia

- Cierre de la saga y sincronización de estados
- Retry Pattern y manejo de reintentos automáticos
- Dead Letter Queues y gestión de mensajes fallidos con Shovel
- Observabilidad de eventos y análisis de headers x-death

Notas de implementación

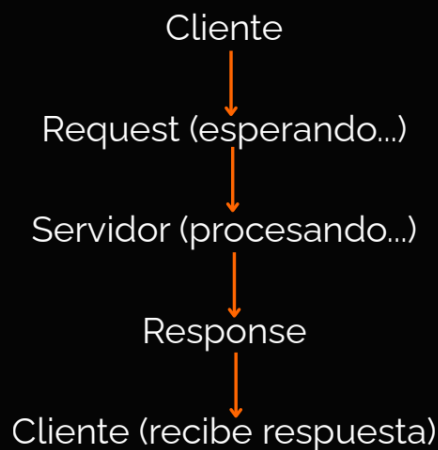
- Persistencia de colas y exchanges configurados como durable
- Gestión de inmutabilidad y resolución del error PRECONDITION_FAILED
- Activación del plugin Shovel para recuperación manual de mensajes desde la UI de RabbitMQ

123. RabbitMQ parte 1

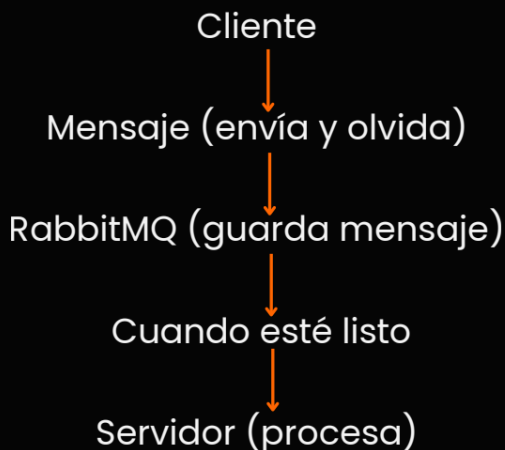
➔ ¿Qué es? Intermediario de mensajes: Permite que diferentes aplicaciones se comuniquen entre sí de forma asíncrona mediante el envío y recepción de mensajes

Síncrona vs Asíncrona

Comunicación síncrona (HTTP)



Comunicación asíncrona (RabbitMQ)



➔ Protocolo AMQP: Advanced Message Queuing Protocol: RabbitMQ NO usa HTTP como el resto de la web. Porque en microservicios la velocidad lo es todo y AMQP no pierde el tiempo saludando ni enviando cabeceras como lo hace http. AMQP envía el mensaje empaquetado en binario, la red lo transporta muy rápido y el receptor lo entiende al instante.

➔ Protocolo para mensajería:

Puerto: 5672.

Lenguajes: Java, Python, Node.js, C#, Go, Ruby, etc.

➔ AMQP vs HTTP:

- Eficiencia Binaria.

- Multiplexación (Canales): En http, se construye una carretera para un solo auto, luego se destruye (se abre una conexión TCP por cada petición). RabbitMQ construye una autopista permanente con miles de carriles (viajes en paralelo).

- Garantías de Entrega: Protocolo asegura que el mensaje no se borre de la cola hasta que el consumidor confirme que ya procesó el mensaje.

124. RabbitMQ parte 2

Arquitectura de RabbitMQ

Componentes Principales

Producer (Productor)



La aplicación que envía mensajes a RabbitMQ.

Exchange (Intercambiador)



Recibe mensajes del Producer y los enruta a las colas según reglas.

Queue (Cola)



Almacena los mensajes hasta que un Consumer los procese.

Consumer (Consumidor)



La aplicación que recibe y procesa mensajes de las colas.

→ Tipos de Exchange: El Exchange decide cómo enrutar los mensajes a las colas.

Direct Exchange

Enruta mensajes a colas con una routing key exacta.

Fanout Exchange

Enruta mensajes a todas las colas conectadas.

Topic Exchange

Enruta con patrones (wildcards).

Headers Exchange

Enruta según headers del mensaje.

→ Patrón

Patrón Publisher/Subscriber



Publisher
(Order Service)



RabbitMQ
(Message broker)



Email Service



Analytics Service

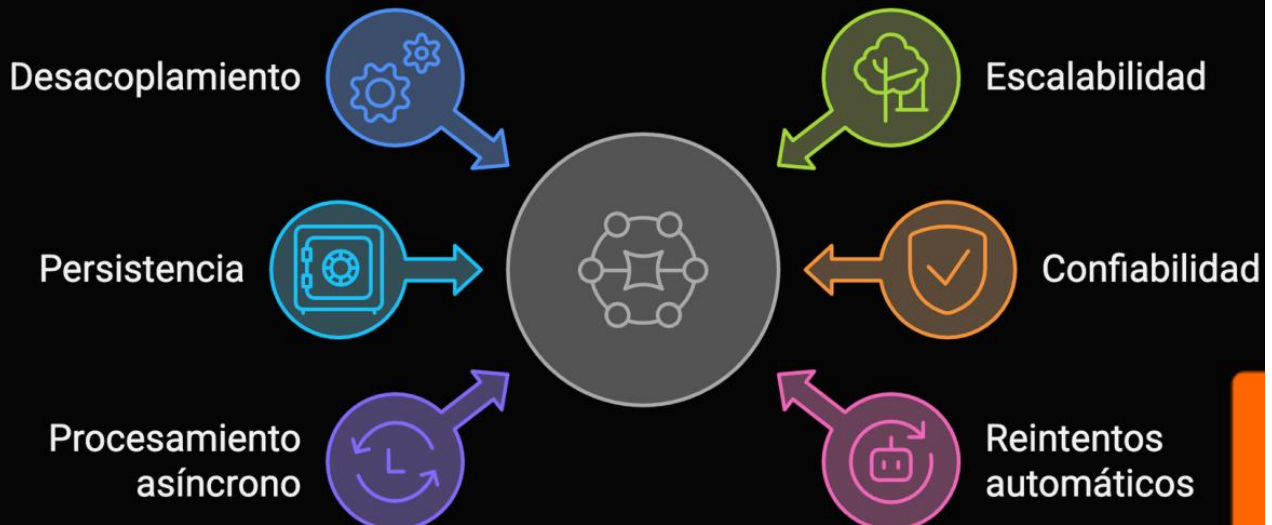


Inventory Service

Ventajas

- Desacoplamiento.
- Escalabilidad.
- Flexibilidad.
- Tolerancia a fallos.

Beneficios de RabbitMQ



125. HTTP y RabbitMQ

➔ ¿Cuándo usar http o RabbitMQ? Pregunta, ¿Necesito la respuesta ya?

** Http: Inmediatez

** RabbitMQ: Estabilidad. Cuando se quiere proteger al sistema en caídas y lentitud en tareas que no urgen.

126. Integrando RabbitMQ al Docker compose

<https://gist.github.com/gchaldu/9ce0ce4946a09c66305d3957e5034207>

```
Project v
├── microservices-ecommerce
│   ├── .idea
│   ├── api-gateway
│   ├── config-data
│   ├── config-repo-cache
│   ├── config-server
│   ├── discovery-server
│   ├── inventory-service
│   ├── notification-service
│   ├── order-service
│   ├── product-service
│   ├── Arquitectura_de_Microservicios_parte_1.pdf
│   └── docker-compose.yml
├── seccion_8_resiliencia_lmites_de_la_sincronia_en...
├── External Libraries
└── Scratches and Consoles

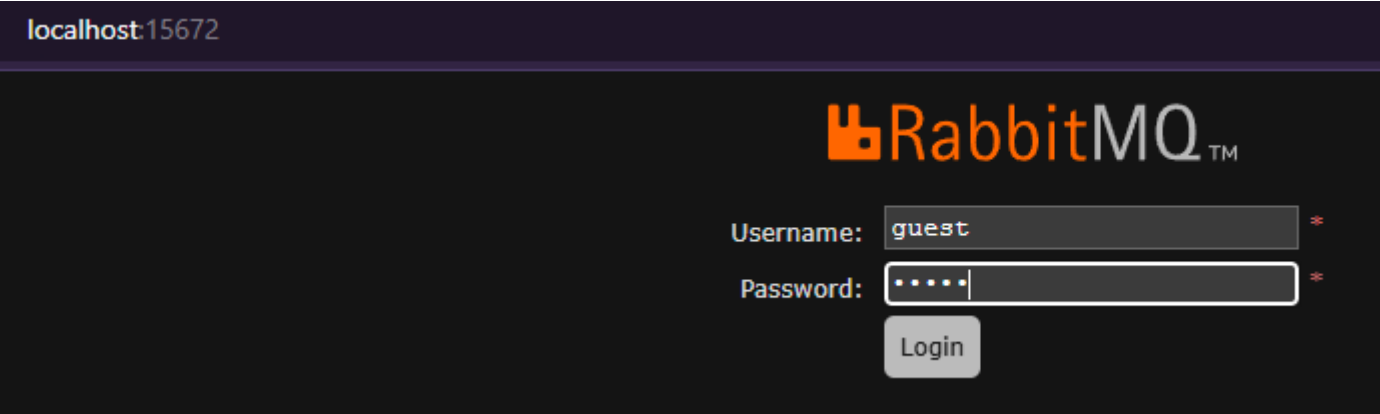
docker-compose.yml
1  services:
2
3  # 1. MongoDB Service
4  mongo:
5    image: mongo:4.2
6    container_name: mongo
7    command: start-dev # Modo desarrollo (rápido y sin HTTPS obligatorio)
8    environment:
9      MONGO_INITDB_ROOT_USERNAME: root
10     MONGO_INITDB_ROOT_PASSWORD: password
11     # Credenciales del Super Admin (Para entrar a la consola ya mismo)
12     MONGO_INITDB_USERNAME: admin
13     MONGO_INITDB_PASSWORD: admin
14    ports:
15      - "27020:27020"
16    depends_on:
17      - mongo-init
18
19 # 2. Keycloak Service
20 keycloak:
21   image: quay.io/keycloak/keycloak:24.0.1
22   container_name: keycloak
23   command: start-dev # Modo desarrollo (rápido y sin HTTPS obligatorio)
24   environment:
25     KC_DB: postgres
26     KC_DB_URL: jdbc:postgresql://keycloak-db:5432/keycloak-db
27     KC_DB_USERNAME: keycloak
28     KC_DB_PASSWORD: password
29     # Credenciales del Super Admin (Para entrar a la consola ya mismo)
30     KEYCLOAK_ADMIN: admin
31     KEYCLOAK_ADMIN_PASSWORD: admin
32   ports:
33     - "8080:8080"
34   depends_on:
35     - keycloak-db
36
37 # RabbitMQ
38 rabbitmq:
39   image: rabbitmq:4.2-management # Version
40   container_name: rabbitmq # Nombre del contenedor (docker desktop)
41   ports:
42     - "5672:5672" # Puerto para comunicación entre microservicios
43     - "15672:15672" # Puerto para la consola de gestión web
44   environment:
45     RABBITMQ_DEFAULT_USER: guest
46     RABBITMQ_DEFAULT_PASS: guest
47   volumes:
48     - rabbitmq_data:/var/lib/rabbitmq # <--- ¡La persistencia!
49
50 volumes:
51   mongo_data: # Declaramos el volumen persistente para product
52   inventory_data: # Declaramos el volumen persistente para inventory
53   order_data: # Declaramos el volumen persistente para orders
54   keycloak_data: # Declaramos el volumen persistente para la seguridad
55   rabbitmq_data: # Declaramos el volumen persistente para rabbitmq
```

➔Levantamos el servicio docker:

```
USER@DESKTOP-7UFJ8GM MINGW64 /d/new_projects/java/microservices-ecommerce (main)
$ docker compose up -d
```

☐		Name	Container ID	Image
☐	○	vigilant_gould	a0459194adcf	hello-world
☐	▼ ●	microservices-ecommerce	-	-
☐	●	keycloak	206fa02ca58d	keycloak/keycloak:24.0.1
☐	●	keycloak-db	b54e7ce1a84b	postgres:16-alpine
☐	●	order-db	f4af02758701	postgres:16-alpine
☐	●	mongodb	249e7edeea34	mongo:7.0.4
☐	●	inventory-db	628396d7d1e8	mysql:8
☐	●	rabbitmq	f41f93031ba0	rabbitmq:4.2-management

Luego ir a <http://localhost:15672/>



localhost:15672/#/

RabbitMQ™

RabbitMQ 4.2.9 Erlang 27.3.4.14

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

Overview

▼ Totals

Queued messages

last minute

?

Currently idle

Message rates

last minute

?

Currently idle

Global counts

?

Connections: 0

Channels: 0

Exchanges: 8

Queues: 0

Consumers: 0

▼ Nodes

Name	File descriptors ?	Erlang processes	Memory ?	Disk space	Uptime	Cores	Info	Reset stats
rabbit@f41f93031ba0	36 1048576 available	427 1048576 available	171 MiB 4.6 GiB high watermark	948 GiB 48 MiB low watermark	4m 44s	20	basic 2 rss	This node All nodes

► Churn statistics

► Ports and contexts

► Export definitions

► Import definitions

HTTP API

Documentation

Tutorials

New releases

Commercial edition

Commercial support

Discussions

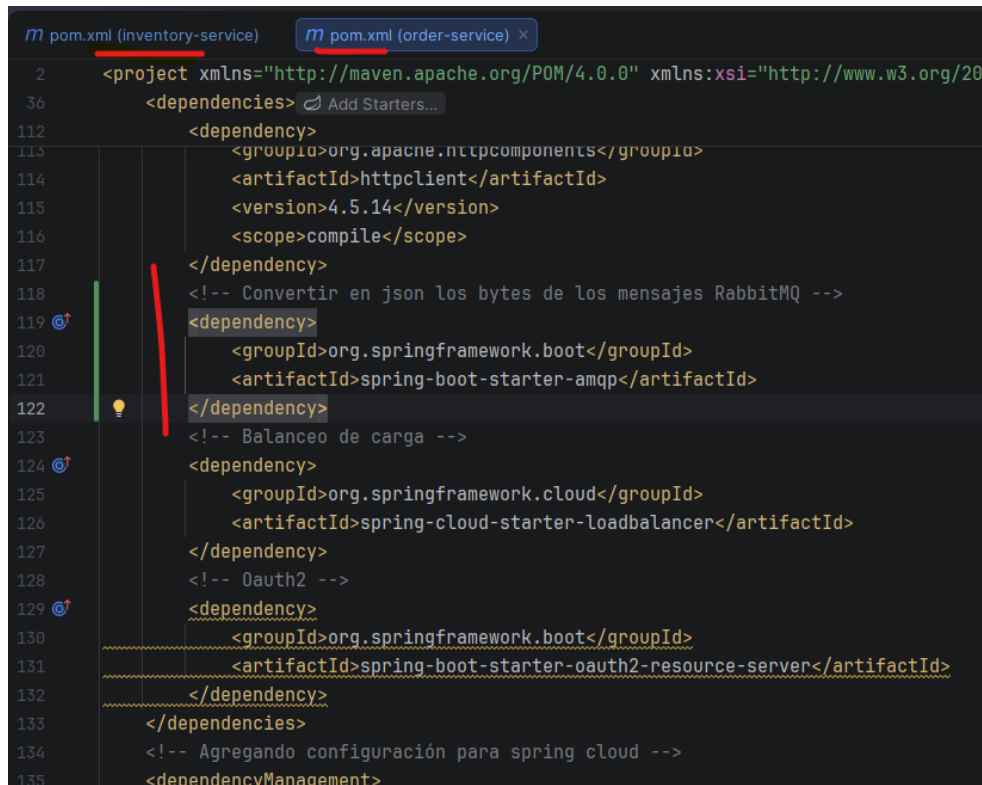
Discord

Plugins

GitHub

127. Configuración JSON (Adiós a los bytes)

→ Spring Boot envía los mensajes como bytes. Para que todo viaje como json, se usa MessageConverter. Para ello se agrega dependencia en inventory-service y order-service:



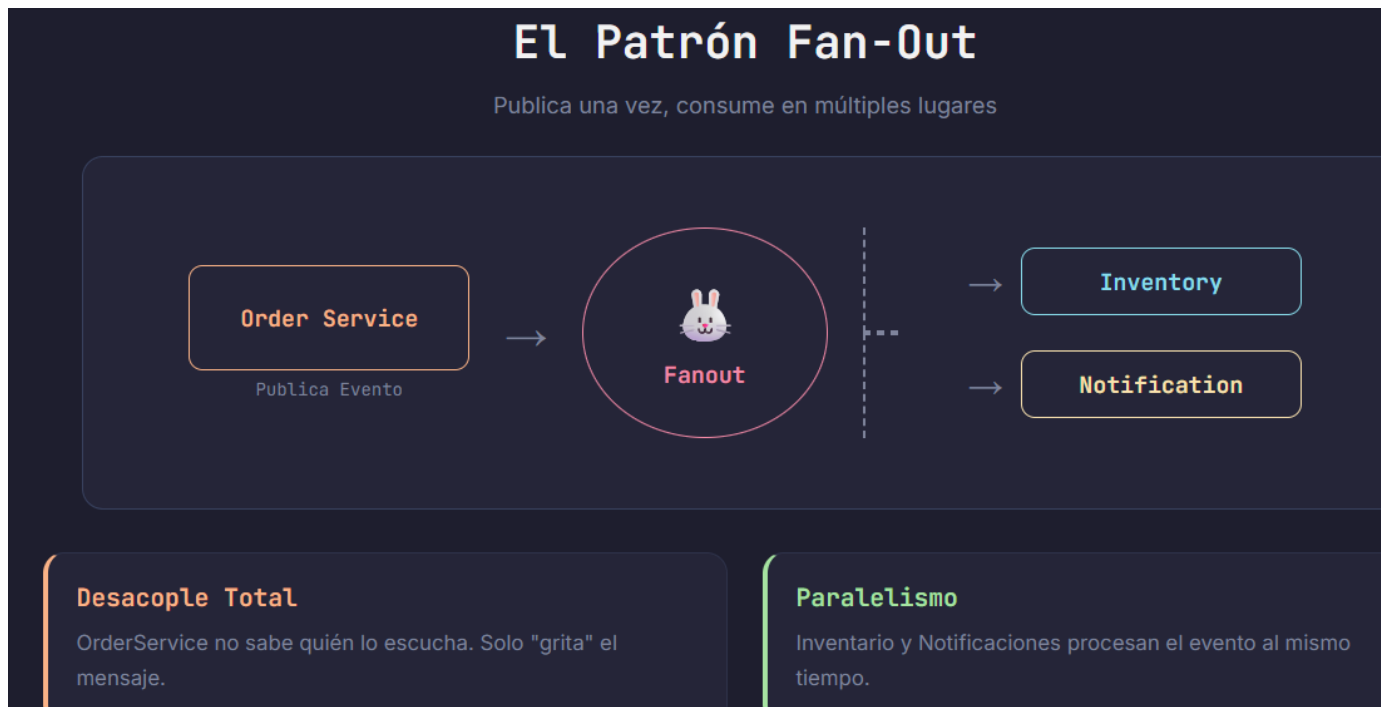
```
m pom.xml (inventory-service) m pom.xml (order-service) x
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/20
36 <dependencies> Add Starters...
112 <dependency>
113 <groupId>org.apache.httpcomponents</groupId>
114 <artifactId>httpclient</artifactId>
115 <version>4.5.14</version>
116 <scope>compile</scope>
117 </dependency>
118 <!-- Convertir en json los bytes de los mensajes RabbitMQ -->
119 <dependency>
120 <groupId>org.springframework.boot</groupId>
121 <artifactId>spring-boot-starter-amqp</artifactId>
122 </dependency>
123 <!-- Balanceo de carga -->
124 <dependency>
125 <groupId>org.springframework.cloud</groupId>
126 <artifactId>spring-cloud-starter-loadbalancer</artifactId>
127 </dependency>
128 <!-- OAuth2 -->
129 <dependency>
130 <groupId>org.springframework.boot</groupId>
131 <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
132 </dependency>
133 </dependencies>
134 <!-- Agregando configuración para spring cloud -->
135 <dependencyManagement>
```

→ Crear clase de configuración en inventory-service:



```
© inventory_service\...\RabbitMQConfig.java x © notification_service\...\RabbitMQConfig.java © order_service\...\RabbitMQConfig.java
1 package com.ecommerce.inventory_service.config;
2
3 import org.springframework.amqp.support.converter.JacksonJsonMessageConverter;
4 import org.springframework.amqp.support.converter.MessageConverter;
5 import org.springframework.context.annotation.Bean;
6 import org.springframework.context.annotation.Configuration;
7
8 @Configuration
9 public class RabbitMQConfig {
10
11     /* Bean para convertir la serialización java a Jackson
12     para convertir los mensajes a json */
13     @Bean
14     public MessageConverter messageConverter() {
15         return new JacksonJsonMessageConverter();
16     }
17 }
18
```

128. Definiendo el evento (El contrato completo)



→ Creando paquete event en order-service donde se habrá un record (inmutables) dentro de otro record (Evento de pedido realizado, OrderPlaceEvent:

```
1 package com.ecommerce.order_service.event;
2
3 import java.util.List;
4
5 public record OrderPlaceEvent(String orderNumber, String email, List<OrderItemEvent> items) {
6     public record OrderItemEvent(String sku, String price, Integer quantity) {
7     }
8 }
9
```

Record es un tipo especial de clase diseñada específicamente para contener datos inmutables. La diferencia principal es que los *records* son automáticos y concisos, ya que generan por detrás el constructor, los *getters*, *equals()*, *hashCode()* y *toString()* sin escribir código repetitivo.

→ Copiar la carpeta event y pegarlo en inventory-service y notification-service:

```
1 package com.ecommerce.order_service.event;
2
3 import java.util.List;
4
5 public record OrderPlaceEvent(String orderNumber, String email, List<OrderItemEvent> items) {
6     public record OrderItemEvent(String sku, String price, Integer quantity) {
7     }
8 }
```

¿Por qué necesitamos un evento completo?

Evitando el re-acoplamiento

✗ Evento incompleto

Solo enviamos el ID de la orden

```
{
  "orderNumber": "ORD-123"
}
```

Problema: Inventory tiene que llamar de vuelta a Order-Service. Esto es lento y frágil.

✓ Evento completo

Payload autosuficiente

```
{
  "orderNumber": "ORD-123",
  "email": "user@email.com",
  "items": [
    {"sku": "iphone", "qty": 2}
  ]
}
```

Solución: Inventory tiene todo lo que necesita para trabajar solo.

¿Por qué usar Nested Records?

Estructura limpia y portable

PRINCIPIO 1

Alta Cohesión

El "Item de Evento" **no existe por sí solo**. Depende totalmente del evento padre. Anidarlo refleja esta relación fuerte.

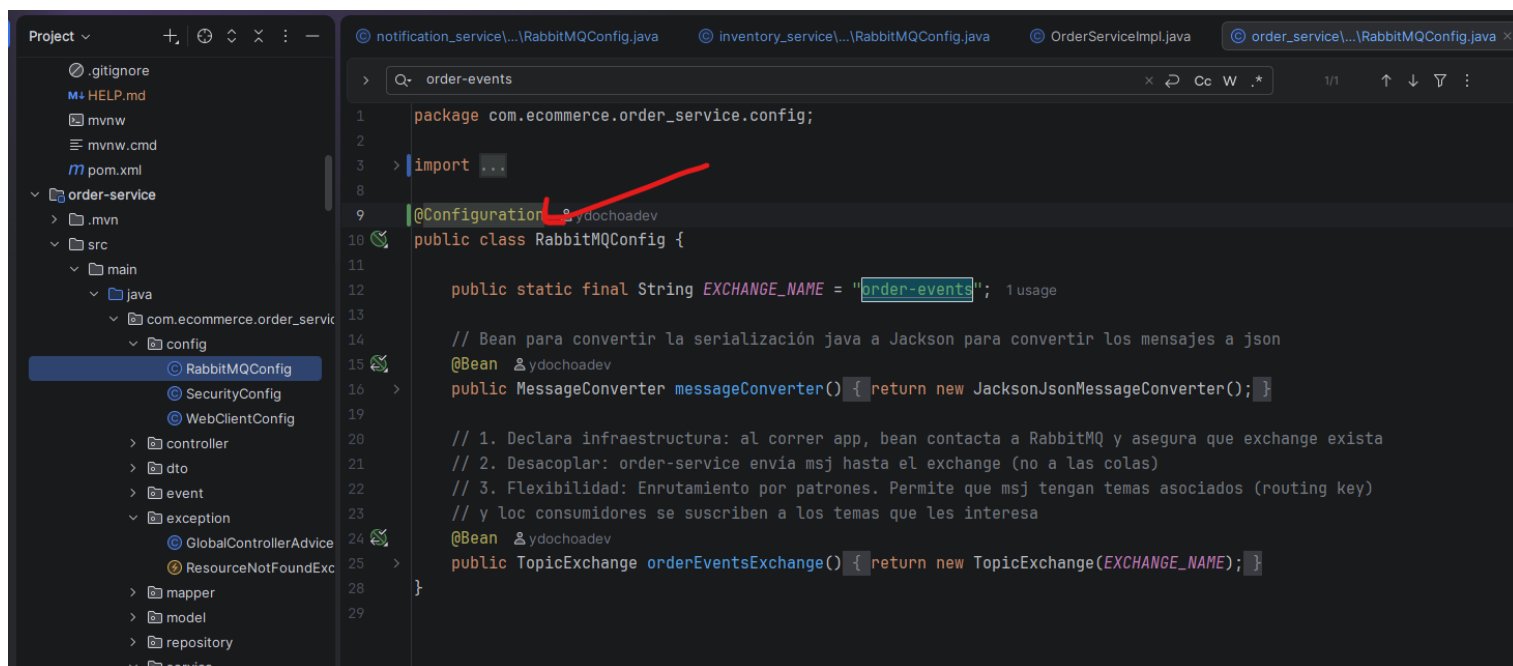
PRINCIPIO 2

Atomicidad

Al tener todo en un solo archivo, garantizamos que la definición del esquema sea **indivisible**. Quien tenga el evento, tiene la estructura completa.

129. Producer (Order Service)

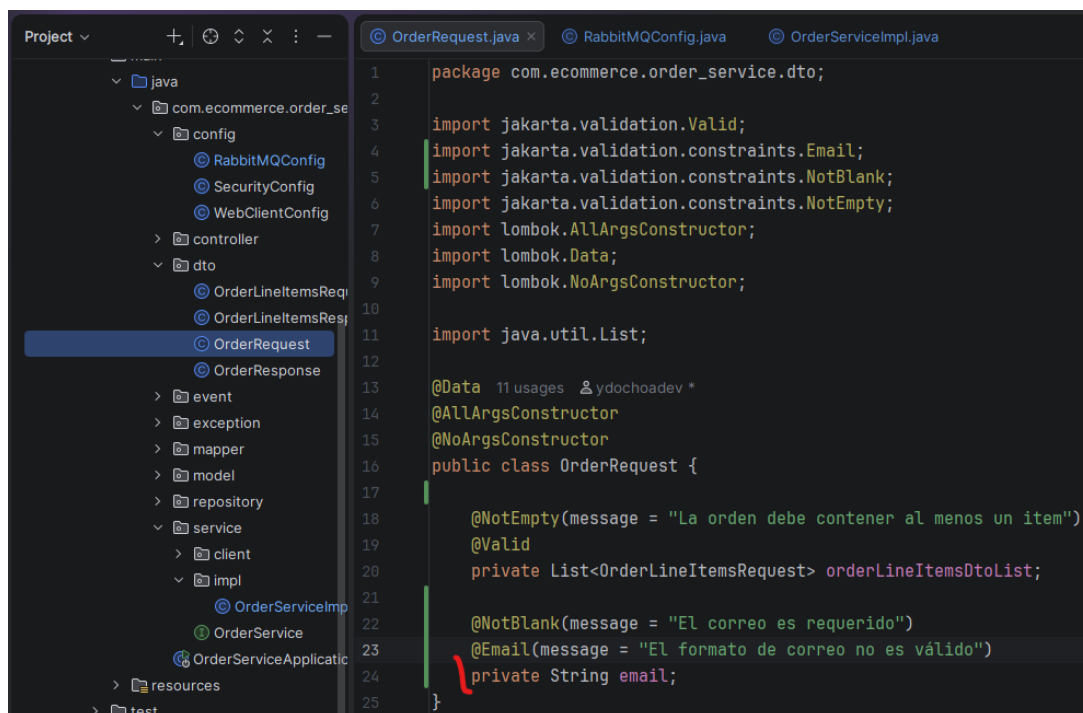
➔ En order-service, configurar “Exchange” (buzón de salida): Antes de enviar algo, es necesario el lugar donde se envían los mensajes, en RabbitMQ no se envían directamente a las colas, si no a un exchange (un intercambiador – ver video 124)



```
1 package com.ecommerce.order_service.config;
2
3 > import ...
4
5 @Configuration
6 public class RabbitMQConfig {
7
8     public static final String EXCHANGE_NAME = "order-events";
9
10    // Bean para convertir la serialización java a Jackson para convertir los mensajes a json
11    @Bean
12    public MessageConverter messageConverter() { return new JacksonJsonMessageConverter(); }
13
14    // 1. Declara infraestructura: al correr app, bean contacta a RabbitMQ y asegura que exchange exista
15    // 2. Desacoplar: order-service envia msj hasta el exchange (no a las colas)
16    // 3. Flexibilidad: Enrutamiento por patrones. Permite que msj tengan temas asociados (routing key)
17    // y los consumidores se suscriben a los temas que les interesa
18    @Bean
19    public TopicExchange orderEventsExchange() { return new TopicExchange(EXCHANGE_NAME); }
20 }
```

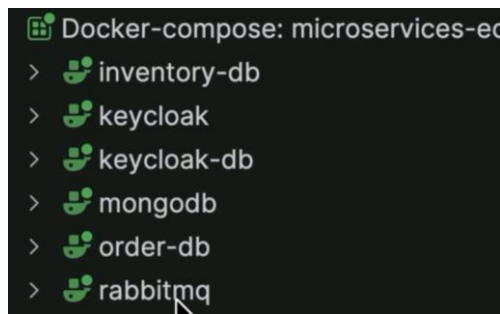
➔ En OrderServiceImpl: Como ya no habrá llamadas Http (inventory-service), ya no se necesita el servicio ni tampoco la configuración CircuitBreaker ni Retry.

** Agregar email al request de order:

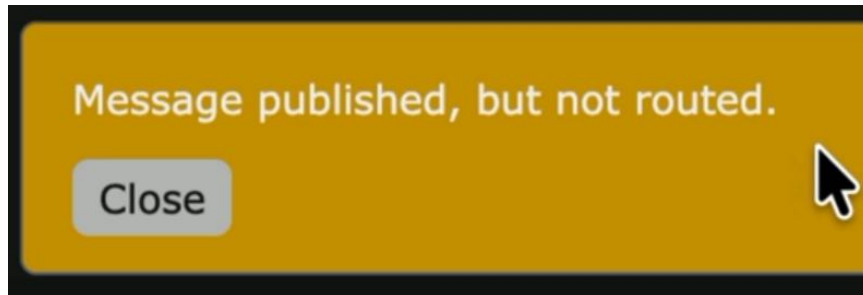


```
1 package com.ecommerce.order_service.dto;
2
3 import jakarta.validation.Valid;
4 import jakarta.validation.constraints.Email;
5 import jakarta.validation.constraints.NotBlank;
6 import jakarta.validation.constraints.NotEmpty;
7 import lombok.AllArgsConstructor;
8 import lombok.Data;
9 import lombok.NoArgsConstructor;
10
11 import java.util.List;
12
13 @Data
14 @AllArgsConstructor
15 @NoArgsConstructor
16 public class OrderRequest {
17
18     @NotEmpty(message = "La orden debe contener al menos un item")
19     @Valid
20     private List<OrderLineItemsRequest> orderLineItemsDtoList;
21
22     @NotBlank(message = "El correo es requerido")
23     @Email(message = "El formato de correo no es válido")
24     private String email;
25 }
```


130. Ejecución y validación Postman & RabbitMQ



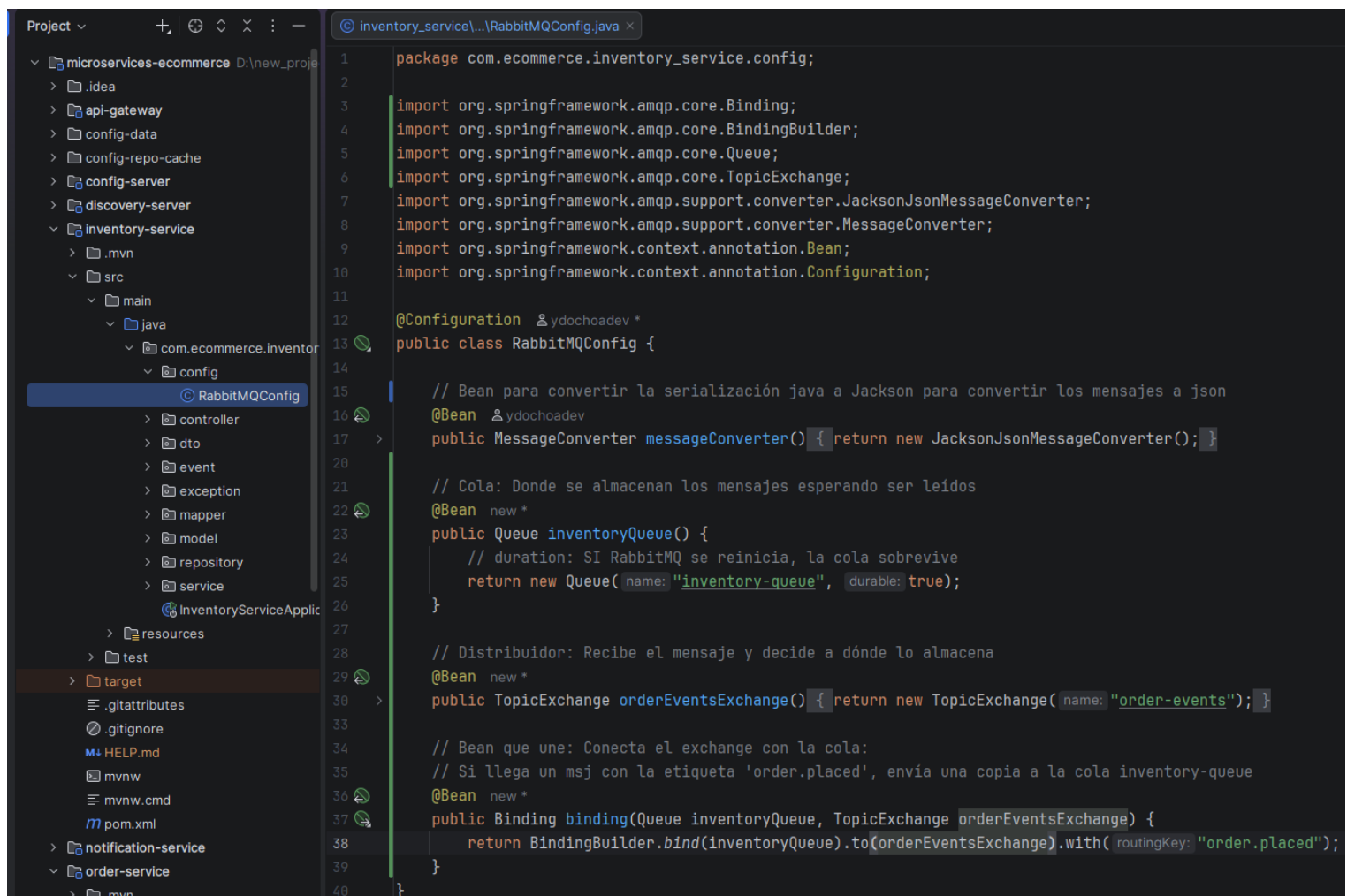
➔ Al enviar una petición desde Bruno en order-service, e ir a rabbitMQ: localhost:15672, sección Exchange (click en Publish message)



Este mensaje sale porque aún no se tiene una cola para almacenar los mensajes; esto hace que se borre el mensaje, ya que los eschanges no guardan mensajes, solo los reparte.

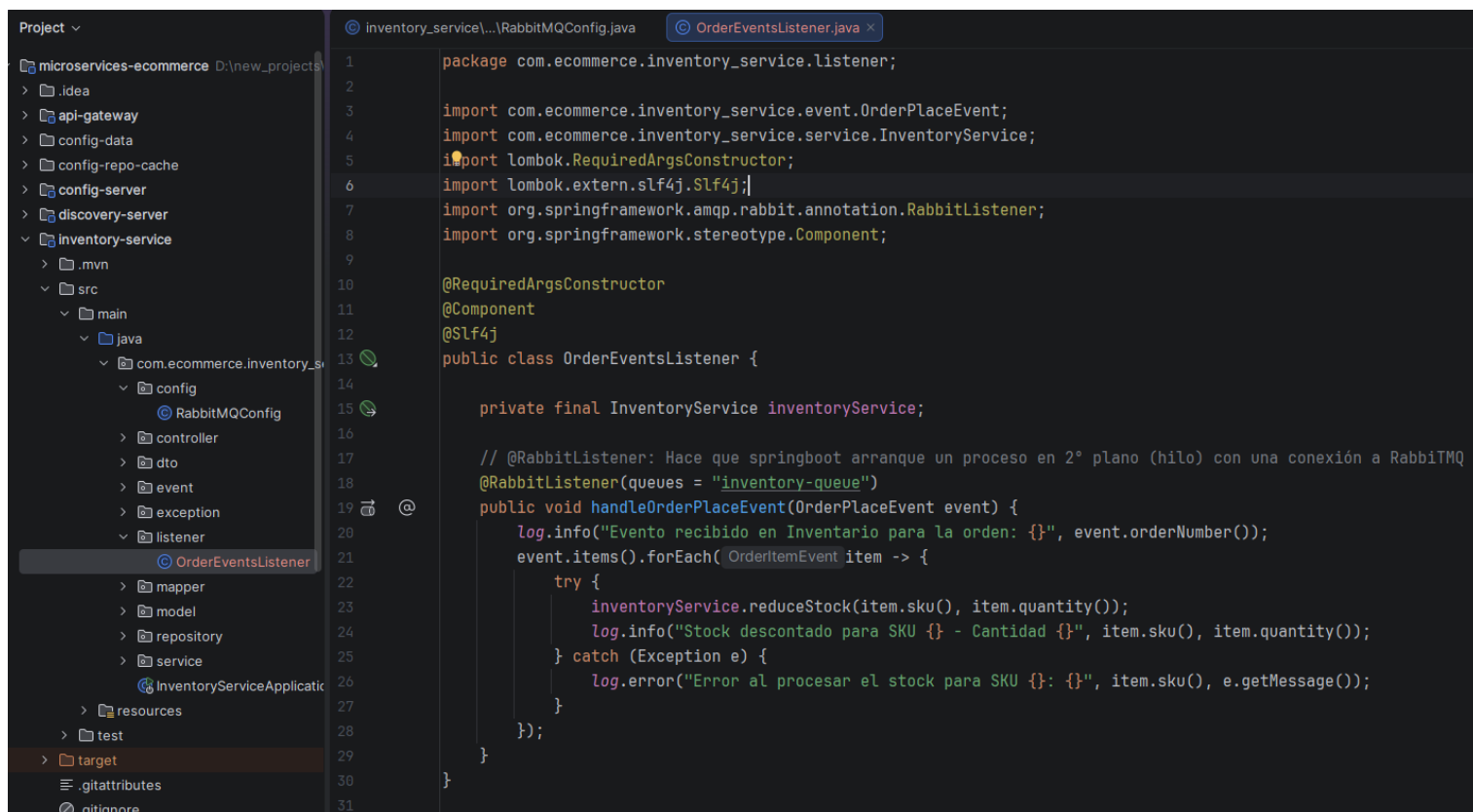
131. Configuración RabbitMQ: (Queue & Binding)

➔ En inventory-service, en la configuración de RabbitMQ:



132. Crear el listener y probar (La escucha)

➔ En inventory-service, crea la clase OrderEventsListener dentro del paquete listener:



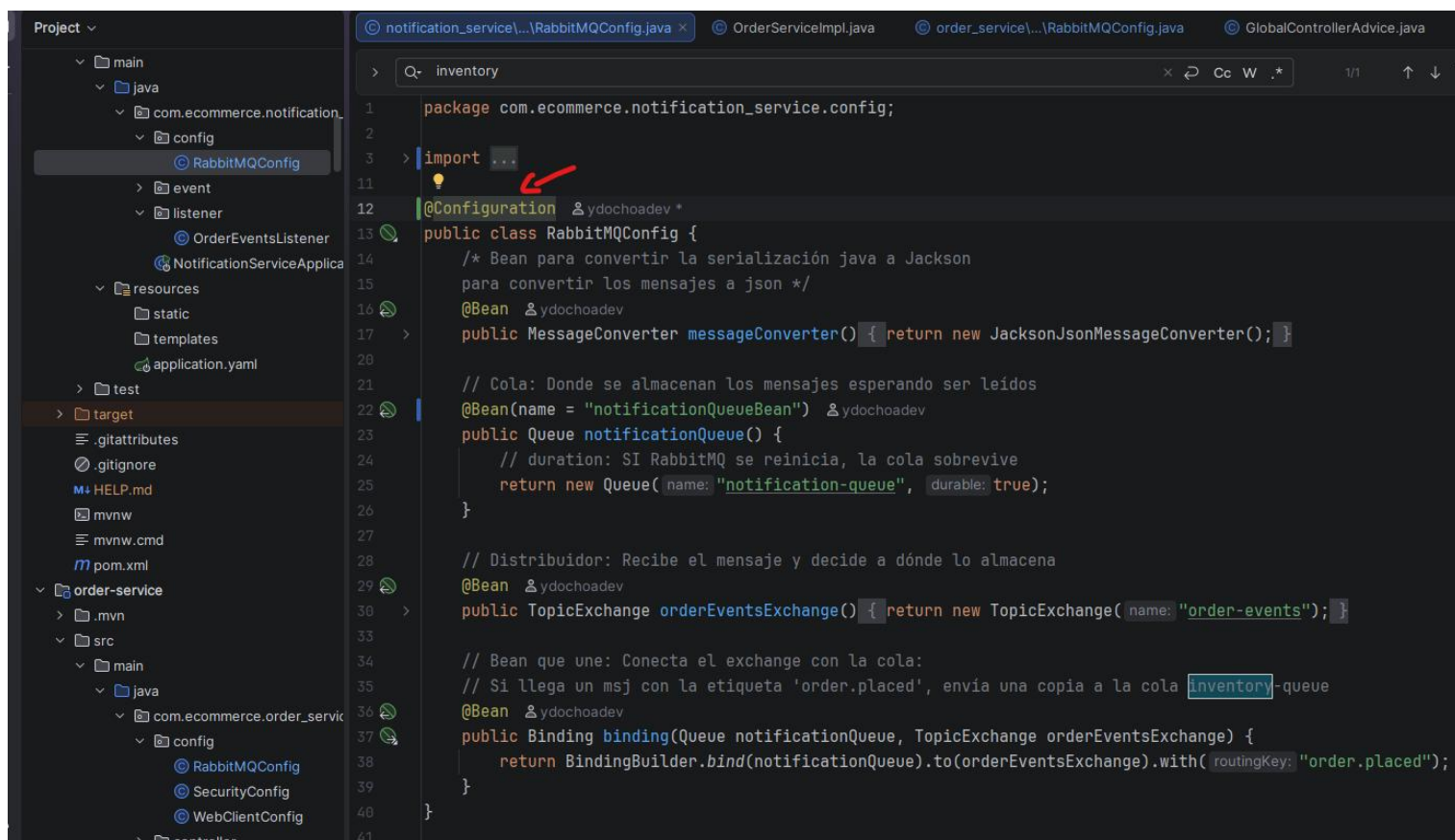
The screenshot shows an IDE with the project structure on the left and the code for `OrderEventsListener.java` in the center. The project structure includes `microservices-e-commerce` with sub-projects like `api-gateway`, `config-data`, `config-repo-cache`, `config-server`, `discovery-server`, and `inventory-service`. The `inventory-service` project has a `src/main/java/com/ecommerce/inventory_service/listener` package. The `OrderEventsListener.java` file is open, showing the following code:

```
1 package com.ecommerce.inventory_service.listener;
2
3 import com.ecommerce.inventory_service.event.OrderPlaceEvent;
4 import com.ecommerce.inventory_service.service.InventoryService;
5 import lombok.RequiredArgsConstructor;
6 import lombok.extern.slf4j.Slf4j;
7 import org.springframework.amqp.rabbit.annotation.RabbitListener;
8 import org.springframework.stereotype.Component;
9
10 @RequiredArgsConstructor
11 @Component
12 @Slf4j
13 public class OrderEventsListener {
14
15     private final InventoryService inventoryService;
16
17     // @RabbitListener: Hace que springboot arranque un proceso en 2º plano (hilo) con una conexión a RabbitMQ
18     @RabbitListener(queues = "inventory-queue")
19     public void handleOrderPlaceEvent(OrderPlaceEvent event) {
20         log.info("Evento recibido en Inventario para la orden: {}", event.orderNumber());
21         event.items().forEach( OrderItemEvent item -> {
22             try {
23                 inventoryService.reduceStock(item.sku(), item.quantity());
24                 log.info("Stock descontado para SKU {} - Cantidad {}", item.sku(), item.quantity());
25             } catch (Exception e) {
26                 log.error("Error al procesar el stock para SKU {}: {}", item.sku(), e.getMessage());
27             }
28         });
29     }
30 }
31
```

133. Fan-Out Completado (Notification service)

➔ En notification-service:

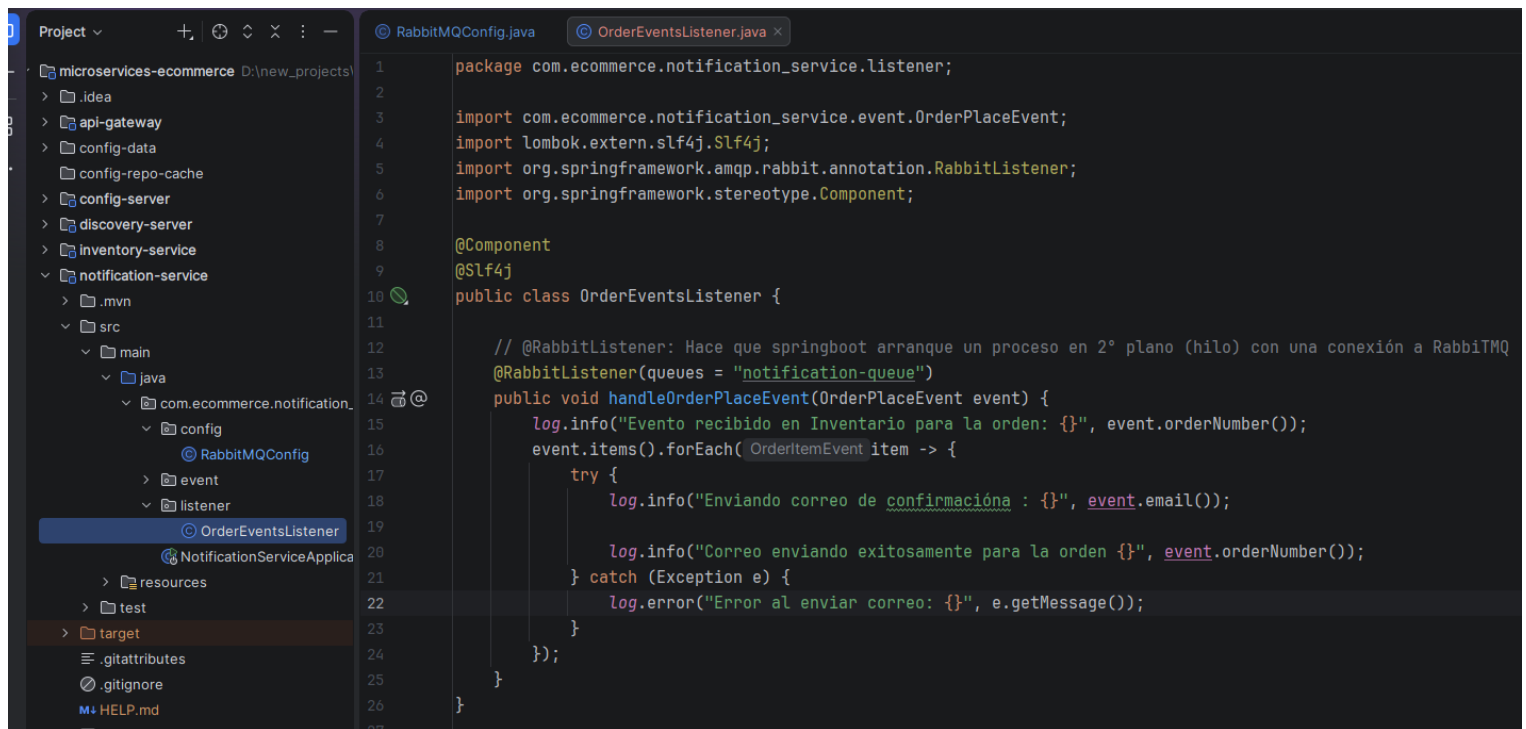
En configuración:



The screenshot shows an IDE with the project structure on the left and the code for `RabbitMQConfig.java` in the center. The project structure includes `notification-service` with sub-projects like `main`, `java`, `com.ecommerce.notification_service`, `config`, `event`, `listener`, `resources`, `static`, `templates`, `application.yaml`, `test`, `target`, `.gitattributes`, `.gitignore`, `HELP.md`, `mvnw`, `mvnw.cmd`, `pom.xml`, and `order-service`. The `notification-service` project has a `src/main/java/com/ecommerce/notification_service/config` package. The `RabbitMQConfig.java` file is open, showing the following code:

```
1 package com.ecommerce.notification_service.config;
2
3 import ...
4
5 @Configuration
6 public class RabbitMQConfig {
7     /* Bean para convertir la serialización java a Jackson
8     para convertir los mensajes a json */
9     @Bean
10     public MessageConverter messageConverter() { return new JacksonJsonMessageConverter(); }
11
12     // Cola: Donde se almacenan los mensajes esperando ser leídos
13     @Bean(name = "notificationQueueBean")
14     public Queue notificationQueue() {
15         // duration: SI RabbitMQ se reinicia, la cola sobrevive
16         return new Queue("notification-queue", durable: true);
17     }
18
19     // Distribuidor: Recibe el mensaje y decide a dónde lo almacena
20     @Bean
21     public TopicExchange orderEventsExchange() { return new TopicExchange("order-events"); }
22
23     // Bean que une: Conecta el exchange con la cola:
24     // Si llega un msj con la etiqueta 'order.placed', envía una copia a la cola 'inventory-queue'
25     @Bean
26     public Binding binding(Queue notificationQueue, TopicExchange orderEventsExchange) {
27         return BindingBuilder.bind(notificationQueue).to(orderEventsExchange).with("order.placed");
28     }
29 }
30
```

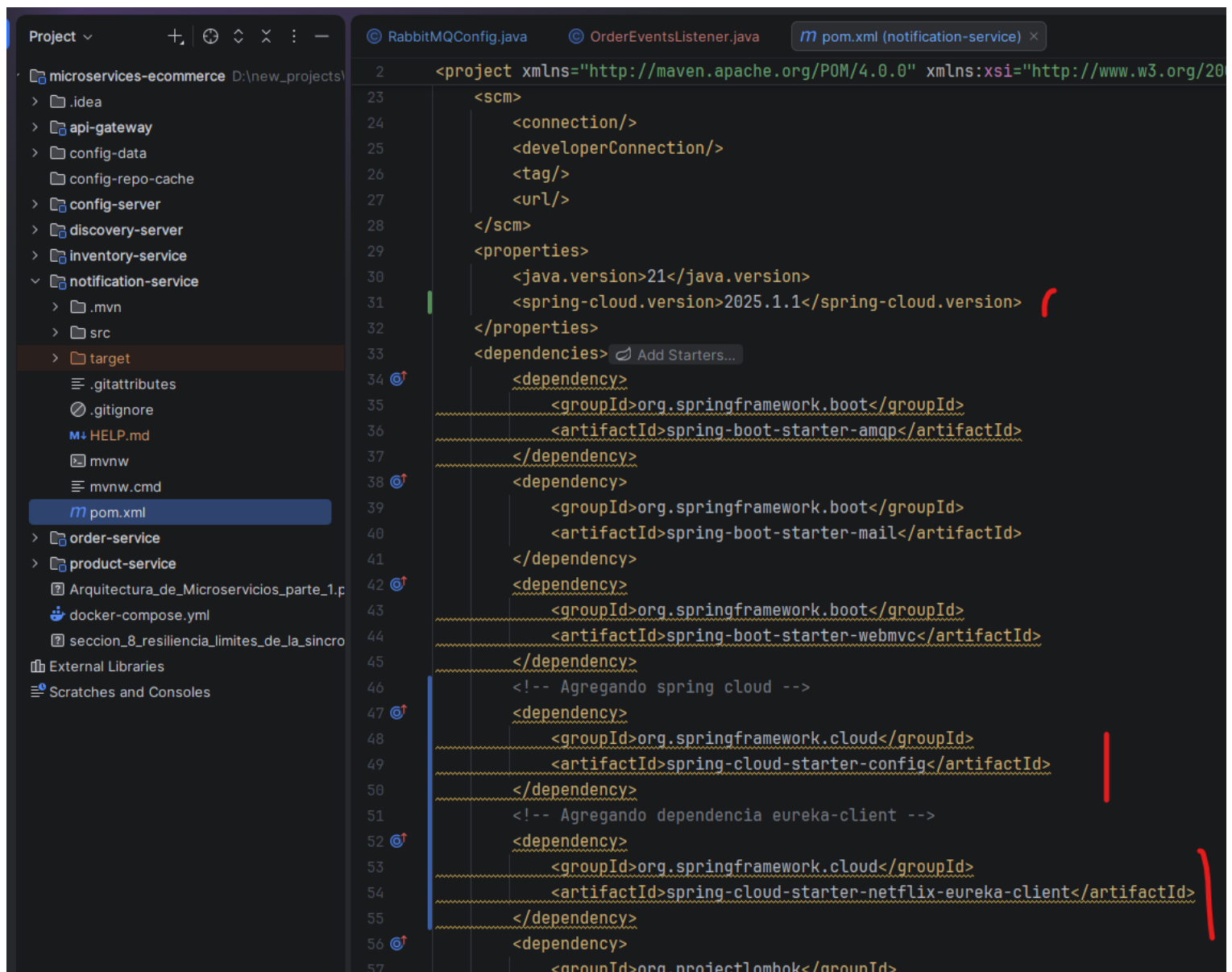

Listener:



```
1 package com.ecommerce.notification_service.listener;
2
3 import com.ecommerce.notification_service.event.OrderPlaceEvent;
4 import lombok.extern.slf4j.Slf4j;
5 import org.springframework.amqp.rabbit.annotation.RabbitListener;
6 import org.springframework.stereotype.Component;
7
8 @Component
9 @Slf4j
10 public class OrderEventsListener {
11
12     // @RabbitListener: Hace que springboot arranque un proceso en 2º plano (hilo) con una conexión a RabbitMQ
13     @RabbitListener(queues = "notification-queue")
14     public void handleOrderPlaceEvent(OrderPlaceEvent event) {
15         log.info("Evento recibido en Inventario para la orden: {}", event.orderNumber());
16         event.items().forEach( OrderItemEvent item -> {
17             try {
18                 log.info("Enviando correo de confirmación : {}", event.email());
19
20                 log.info("Correo enviado exitosamente para la orden {}", event.orderNumber());
21             } catch (Exception e) {
22                 log.error("Error al enviar correo: {}", e.getMessage());
23             }
24         });
25     }
26 }
27
```

➔ Centralizando la configuración de notification-service en el config-data:

** Agregando las dependencias



```
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/20
23 <scm>
24 <connection/>
25 <developerConnection/>
26 <tag/>
27 <url/>
28 </scm>
29 <properties>
30 <java.version>21</java.version>
31 <spring-cloud.version>2025.1.1</spring-cloud.version>
32 </properties>
33 <dependencies>
34 <dependency>
35 <groupId>org.springframework.boot</groupId>
36 <artifactId>spring-boot-starter-amqp</artifactId>
37 </dependency>
38 <dependency>
39 <groupId>org.springframework.boot</groupId>
40 <artifactId>spring-boot-starter-mail</artifactId>
41 </dependency>
42 <dependency>
43 <groupId>org.springframework.boot</groupId>
44 <artifactId>spring-boot-starter-webmvc</artifactId>
45 </dependency>
46 <!-- Agregando spring cloud -->
47 <dependency>
48 <groupId>org.springframework.cloud</groupId>
49 <artifactId>spring-cloud-starter-config</artifactId>
50 </dependency>
51 <!-- Agregando dependencia eureka-client -->
52 <dependency>
53 <groupId>org.springframework.cloud</groupId>
54 <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
55 </dependency>
56 <dependency>
57 <groupId>org.projectlombok</groupId>
```

➔ Agregando nuevo archivo yml para notification-service en config-data:

** commitear

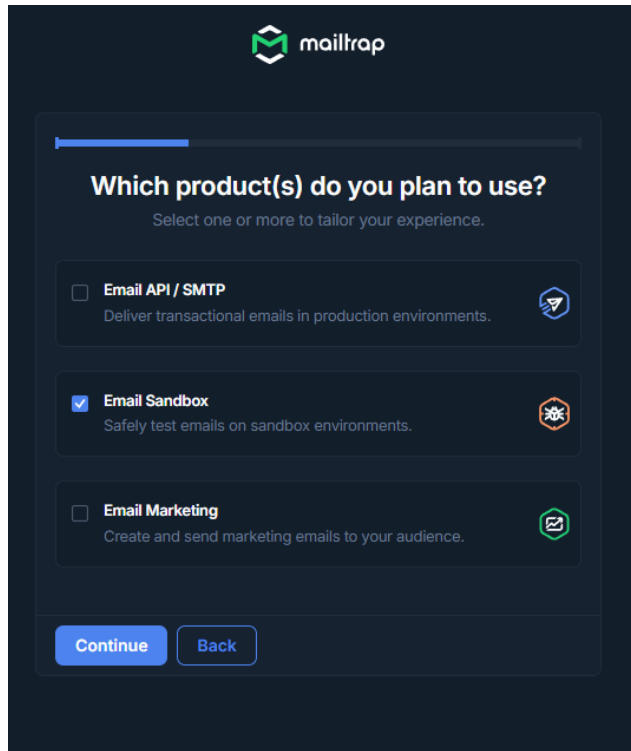
** Agregar configuración en el mismo micro para config-server:

134. Integración JavaMailSender - Envío real

<https://gist.github.com/gchaldou/c6a6bf5aa087754c187b35bbf7ac7691>

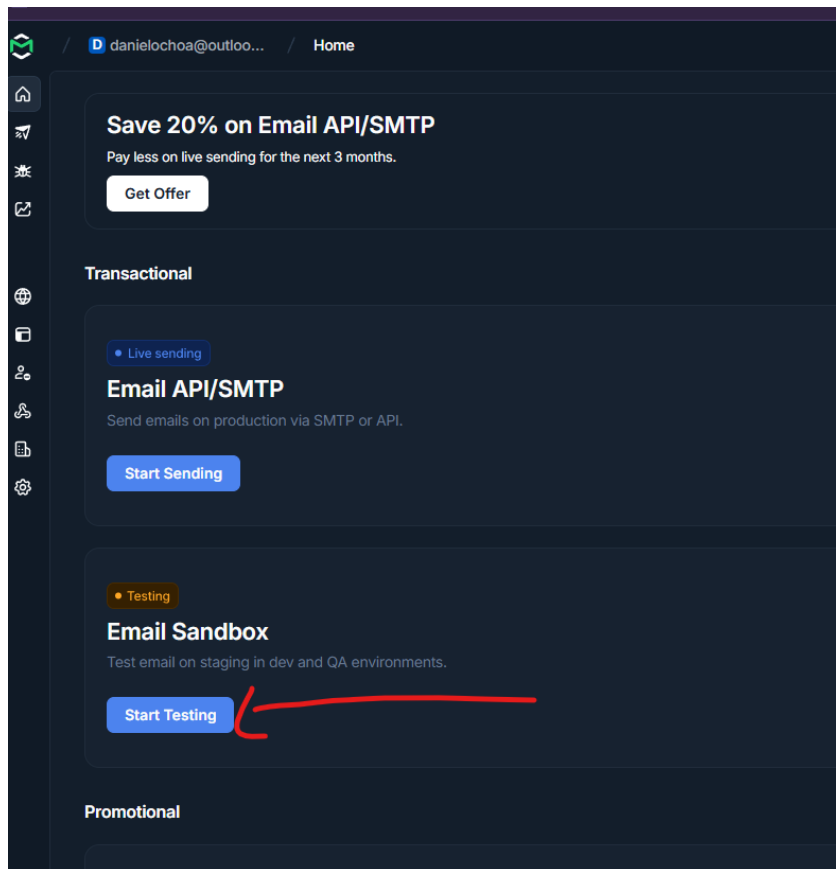
<https://mailtrap.io/>

➔ mailtrap: servidor SMTP falso diseñada para devs. Permite enviar correos desde nuestra aplicación



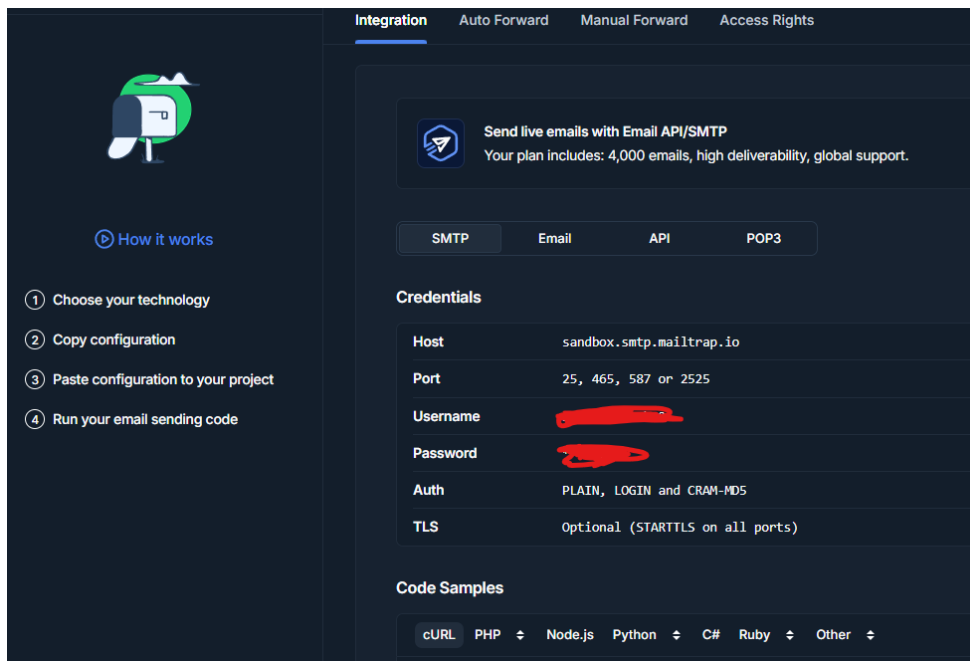
The image shows the Mailtrap onboarding interface. At the top is the Mailtrap logo. Below it, a question asks "Which product(s) do you plan to use?" with the instruction "Select one or more to tailor your experience." There are three options, each with a checkbox and an icon: "Email API / SMTP" (deliver transactional emails), "Email Sandbox" (safely test emails, which is selected with a blue checkmark), and "Email Marketing" (create and send marketing emails). At the bottom are "Continue" and "Back" buttons.

➔ Probar:



The image shows the Mailtrap dashboard. At the top, there's a navigation bar with the Mailtrap logo, a user profile icon, and the text "danielchoa@outloo... / Home". Below the navigation bar is a sidebar with various icons. The main content area has a "Save 20% on Email API/SMTP" offer with a "Get Offer" button. Below that is a "Transactional" section with a "Live sending" tag, "Email API/SMTP" title, and "Start Sending" button. Below that is an "Email Sandbox" section with a "Testing" tag, "Email Sandbox" title, and "Start Testing" button, which is highlighted with a red arrow. At the bottom is a "Promotional" section.

➔ Llevar el user y pass hacia la configuración de la app:



Integration Auto Forward Manual Forward Access Rights

Send live emails with Email API/SMTP
Your plan includes: 4,000 emails, high deliverability, global support.

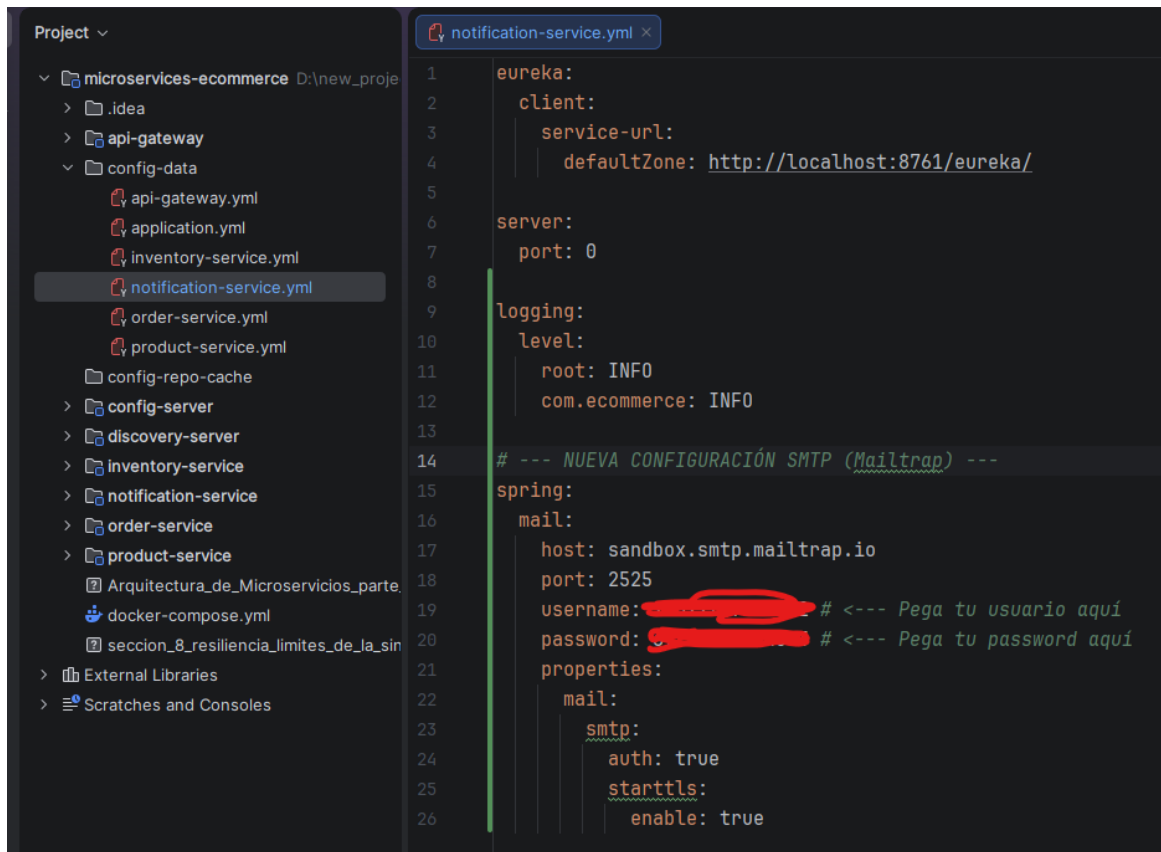
SMTP Email API POP3

Credentials

Host	sandbox.smtp.mailtrap.io
Port	25, 465, 587 or 2525
Username	[REDACTED]
Password	[REDACTED]
Auth	PLAIN, LOGIN and CRAM-MD5
TLS	Optional (STARTTLS on all ports)

Code Samples

cURL PHP Node.js Python C# Ruby Other



Project

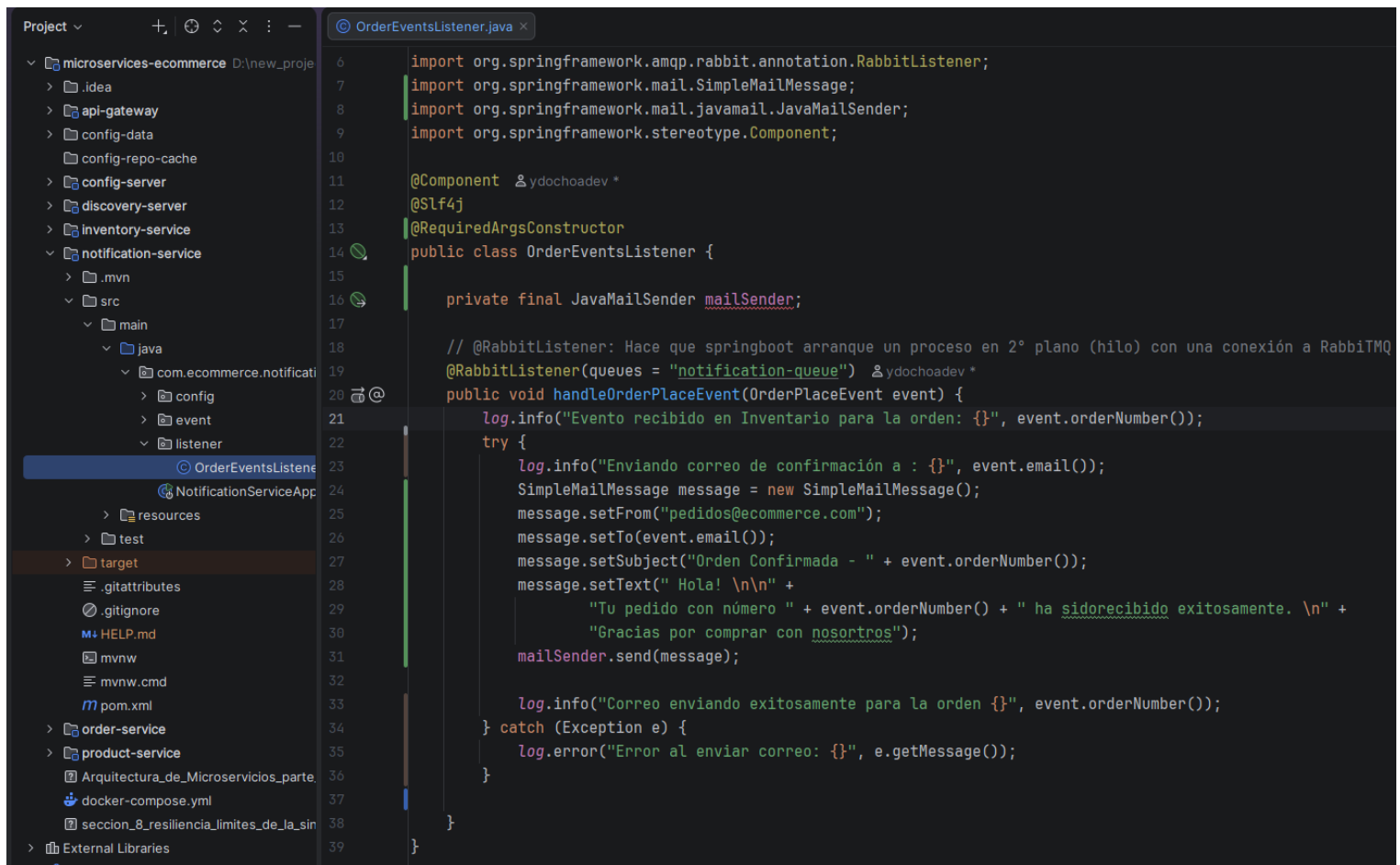
- microservices-ecommerce D:\new_proje
 - .idea
 - api-gateway
 - config-data
 - api-gateway.yml
 - application.yml
 - inventory-service.yml
 - notification-service.yml
 - order-service.yml
 - product-service.yml
 - config-repo-cache
 - config-server
 - discovery-server
 - inventory-service
 - notification-service
 - order-service
 - product-service
 - Arquitectura_de_Microservicios_parte
 - docker-compose.yml
 - seccion_8_resiliencia_limites_de_la_sin
- External Libraries
- Scratches and Consoles

notification-service.yml

```
1 eureka:
2   client:
3     service-url:
4       defaultZone: http://localhost:8761/eureka/
5
6 server:
7   port: 0
8
9 logging:
10  level:
11    root: INFO
12    com.ecommerce: INFO
13
14 # --- NUEVA CONFIGURACIÓN SMTP (Mailtrap) ---
15 spring:
16   mail:
17     host: sandbox.smtp.mailtrap.io
18     port: 2525
19     username: [REDACTED] # <--- Pega tu usuario aquí
20     password: [REDACTED] # <--- Pega tu password aquí
21   properties:
22     mail:
23       smtp:
24         auth: true
25         starttls:
26           enable: true
```

** commitear:

➔ En el listener:



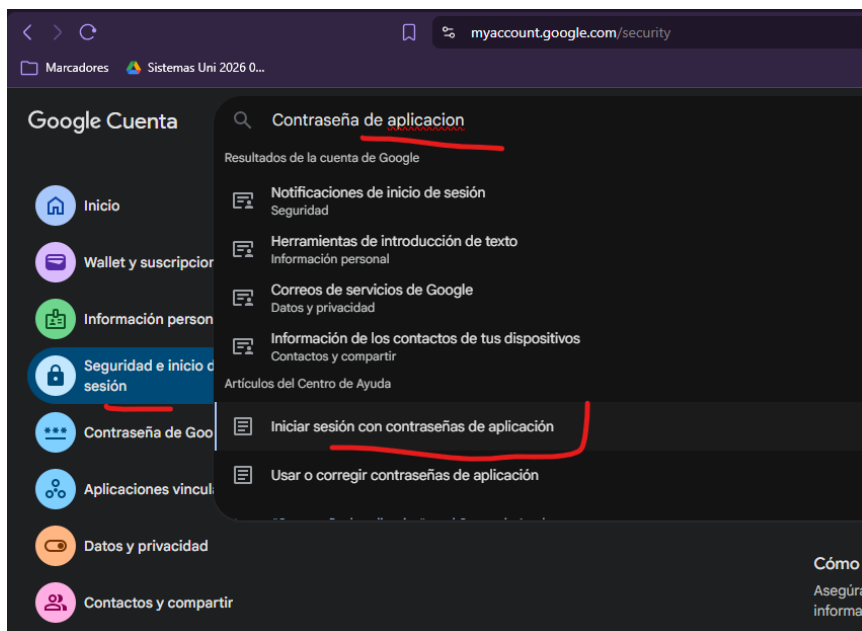
```
6 import org.springframework.amqp.rabbit.annotation.RabbitListener;
7 import org.springframework.mail.SimpleMailMessage;
8 import org.springframework.mail.javamail.JavaMailSender;
9 import org.springframework.stereotype.Component;
10
11 @Component
12 @Slf4j
13 @RequiredArgsConstructor
14 public class OrderEventsListener {
15
16     private final JavaMailSender mailSender;
17
18     // @RabbitListener: Hace que springboot arranque un proceso en 2° plano (hilo) con una conexión a RabbitMQ
19     @RabbitListener(queues = "notification-queue")
20     public void handleOrderPlaceEvent(OrderPlaceEvent event) {
21         log.info("Evento recibido en Inventario para la orden: {}", event.orderNumber());
22         try {
23             log.info("Enviando correo de confirmación a : {}", event.email());
24             SimpleMailMessage message = new SimpleMailMessage();
25             message.setFrom("pedidos@ecommerce.com");
26             message.setTo(event.email());
27             message.setSubject("Orden Confirmada - " + event.orderNumber());
28             message.setText("Ho!a! \n\n" +
29                 "Tu pedido con número " + event.orderNumber() + " ha sido recibido exitosamente. \n" +
30                 "Gracias por comprar con nosotros");
31             mailSender.send(message);
32
33             log.info("Correo enviando exitosamente para la orden {}", event.orderNumber());
34         } catch (Exception e) {
35             log.error("Error al enviar correo: {}", e.getMessage());
36         }
37     }
38 }
39 }
```

➔ Hacer la prueba y ver el mailtrap.

135. Pase a producción (Gmail y Perfiles)

<https://www.google.com/account/about/?hl=es-419>

➔ Crear una configuración nueva para producción.



Ayuda

requieren contraseñas de aplicación. En su lugar, utiliza "Iniciar sesión con Google".

Si la aplicación no ofrece la opción "Iniciar sesión con Google", puedes hacer lo siguiente:

- Usar contraseñas de aplicación
- Cambiar a una aplicación o un dispositivo más seguros

Crear y utilizar contraseñas de aplicación

Importante: Para crear una contraseña de aplicación, debes habilitar la verificación en dos pasos en tu cuenta de Google.

Si utilizas la verificación en dos pasos y aparece el error "Contraseña incorrecta" al iniciar sesión, puedes intentar utilizar una contraseña de aplicación.

[Crea y gestiona contraseñas de aplicación](#) . Es posible que debas iniciar sesión en tu cuenta de Google.

Si has configurado la verificación en dos pasos y no encuentras la opción para añadir una contraseña de aplicación, puede deberse a lo siguiente:

Las contraseñas de aplicación te ayudan a iniciar sesión en tu cuenta de Google en aplicaciones y servicios antiguos que no son compatibles con los estándares de seguridad modernos.

Las contraseñas de aplicación son menos seguras que usar aplicaciones y servicios actualizados que utilicen estándares de seguridad modernos.

Antes de crear una contraseña de aplicación, debes comprobar si tu aplicación la necesita para iniciar sesión.

[Más información](#)

Tus contraseñas de aplicación

Ecommerce-2026

Creada: 4 jul; utilizada por última vez: 4 jul



Para crear una contraseña específica de la aplicación, escribe el nombre de la aplicación a continuación...

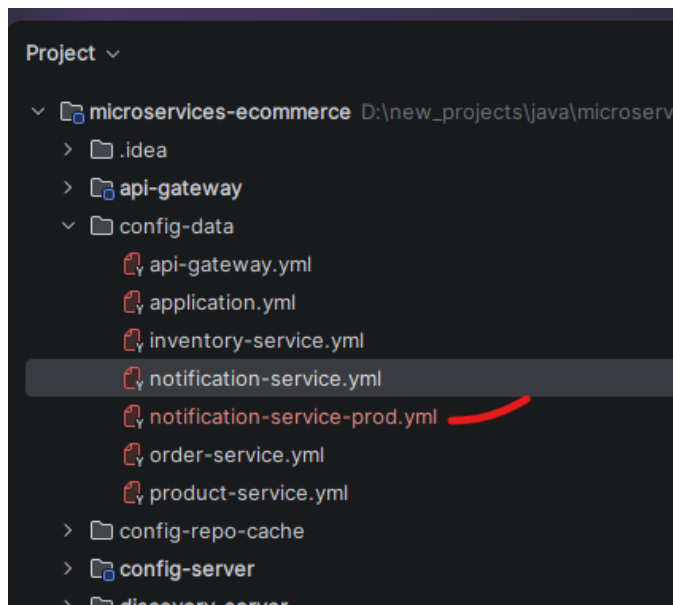
Nombre de la aplicación

ecommerce-notification-service

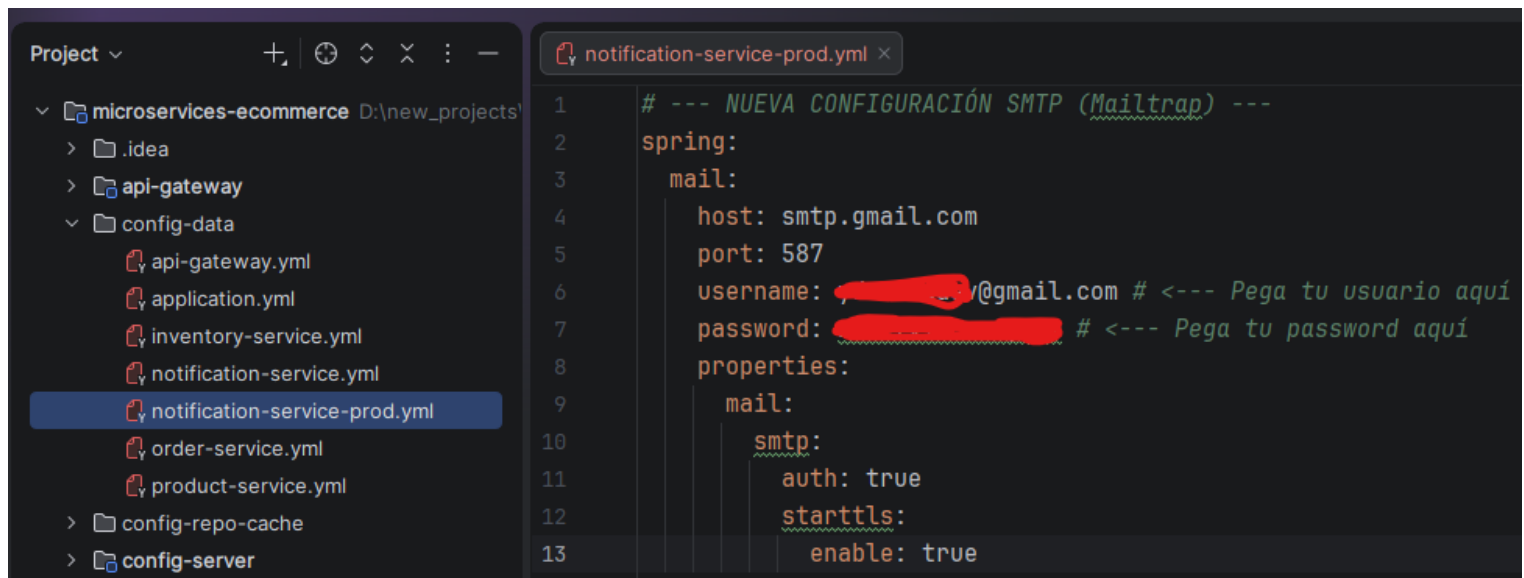
Crear

Obtener la contraseña

➔ Copiar el archivo de configuración de notification para producción:

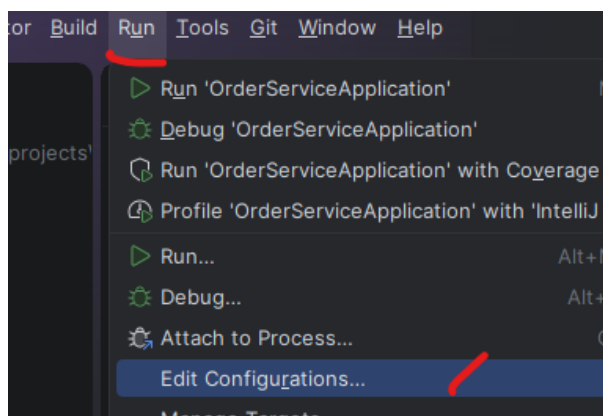


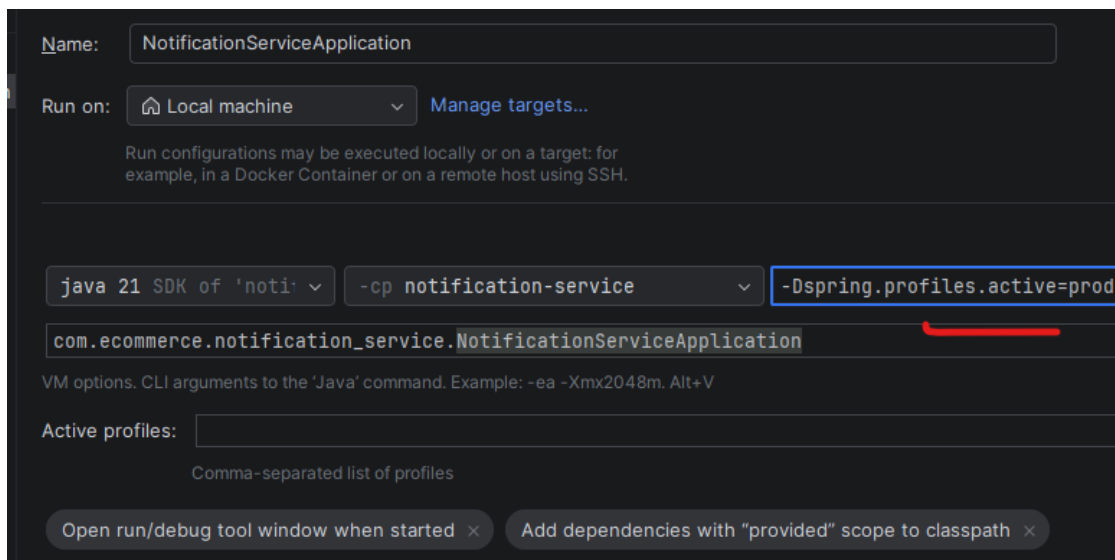
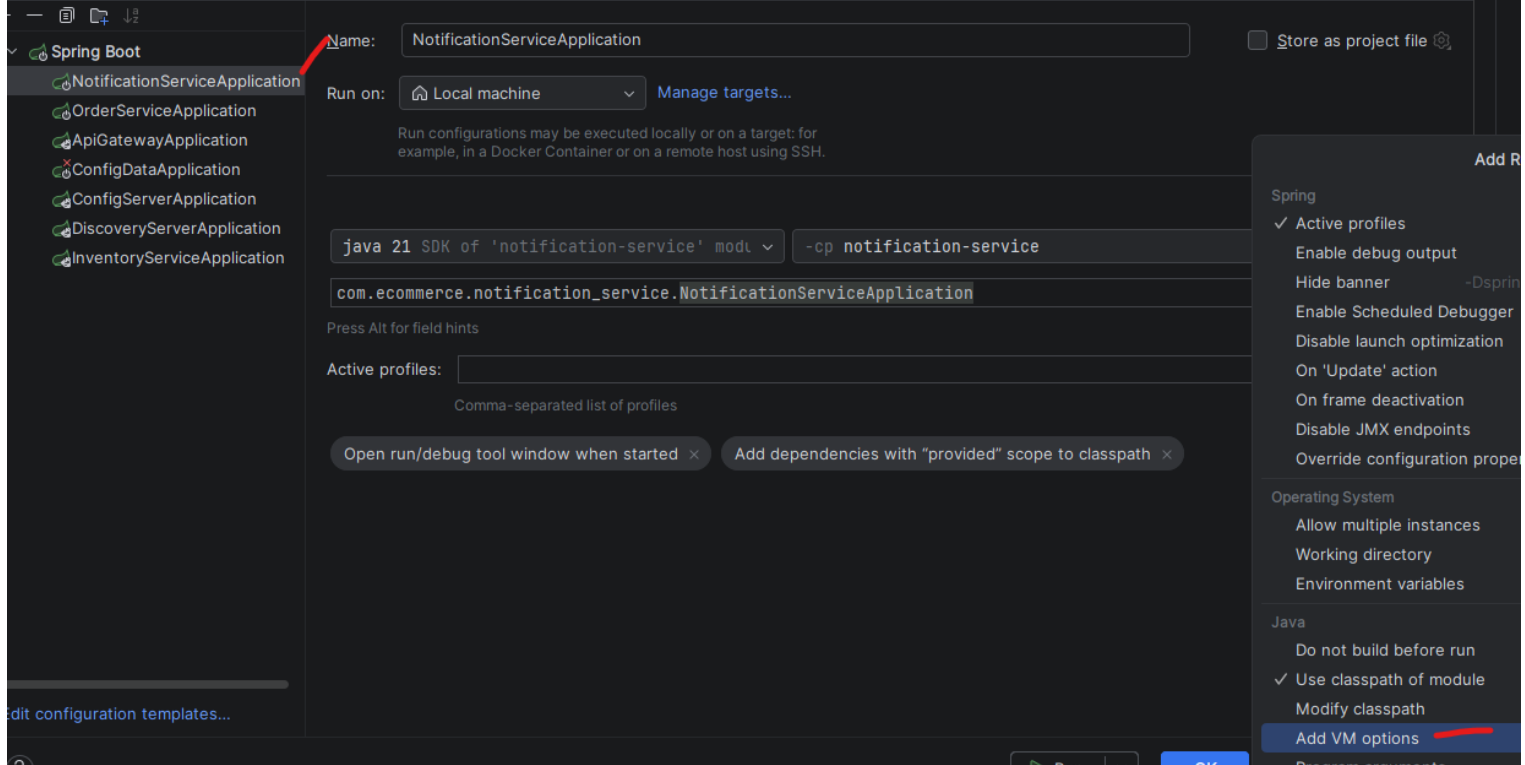
*** Estos 2 archivos, funcionan por “herencia”: Java lee el archivo “notification-service.yml” y valida que el archivo notification-service-prod.yml cambia, tomará el último archivo si estamo en producción.



Committed.

➔ Ahora se debe configurar el perfil o archivo de preferencia que se debe tomar, si el de producción o desarrollo





→ Ejecutar y probar

136. Saga Pattern ¿Qué es?

 ¿Qué es el Saga Pattern?

Una analogía con las vacaciones

Imagina que estás organizando unas vacaciones. Tienes que reservar **tres cosas**:



Vuelo



Hotel



Alquiler de Auto

En un **sistema monolítico**: hacías todo en una sola transacción de base de datos. Si fallaba el auto, la base de datos hacía **Rollback** automático y borraba el vuelo y el hotel mágicamente. **Todo o nada**.

El Problema en Microservicios

No existe el rollback mágico entre servicios

Como cada microservicio tiene su **propia base de datos**, no existe el Rollback mágico entre ellos.

- 1 `OrderService` guarda la orden en su BD...
- 2 Y luego `InventoryService` dice "No hay stock"...
- ✗ `OrderService` **no se entra automáticamente**. La orden se queda ahí, "colgada" y el cliente cree que compró.

⚠ **Resultado:** Datos inconsistentes entre servicios. El cliente ve una orden confirmada que nunca se procesó.

SAGA: Una historia de compensaciones

Si rompes algo, tienes que arreglarlo tú mismo

Definición de Saga

Una Saga es una **secuencia de transacciones locales**. Si una falla, ejecutamos **Transacciones de Compensación** para deshacer lo que hicimos antes.

Saga = "Si rompes algo, tienes que arreglarlo tú mismo."

TRANSACCIÓN LOCAL

Cada servicio hace su trabajo en su propia BD

COMPENSACIÓN

Si algo falla, ejecuto la acción inversa para "deshacer"



Coreografía u Orquestación

Dos formas de implementar Sagas

Existen dos formas de implementar Sagas. En nuestro curso estamos usando **Coreografía** (basada en Eventos), que es la más común con RabbitMQ.

NUESTRO ENFOQUE

1. Coreografía

Es como un **baile donde nadie dirige**. Cada bailarín (microservicio) conoce su paso y reacciona a la música (eventos).

- ▶ **Ventaja:** Desacoplado y rápido
- ▶ **Desventaja:** Si son muchos pasos, el baile se vuelve caótico

ALTERNATIVA

2. Orquestación

Un **director de orquesta** coordina todos los pasos. Un servicio central dice qué hacer y cuándo.

- ▶ **Ventaja:** Flujo centralizado y claro
- ▶ **Desventaja:** Acoplamiento al orquestador

Coreografía: Paso a Paso

Cómo funciona en nuestro e-commerce

1 Paso 1: Orden Creada

`OrderService` crea la orden y grita: "¡Orden Creada!" (Evento)

2 Paso 2: Verificación de Stock

`InventoryService` escucha y verifica stock:

- ▶ Si hay stock: Grita "¡Stock Reservado!" ✓
- ▶ Si NO hay stock (Fallo): Grita "¡Fallo de Inventario!" ✗

(Este es el inicio de la compensación)

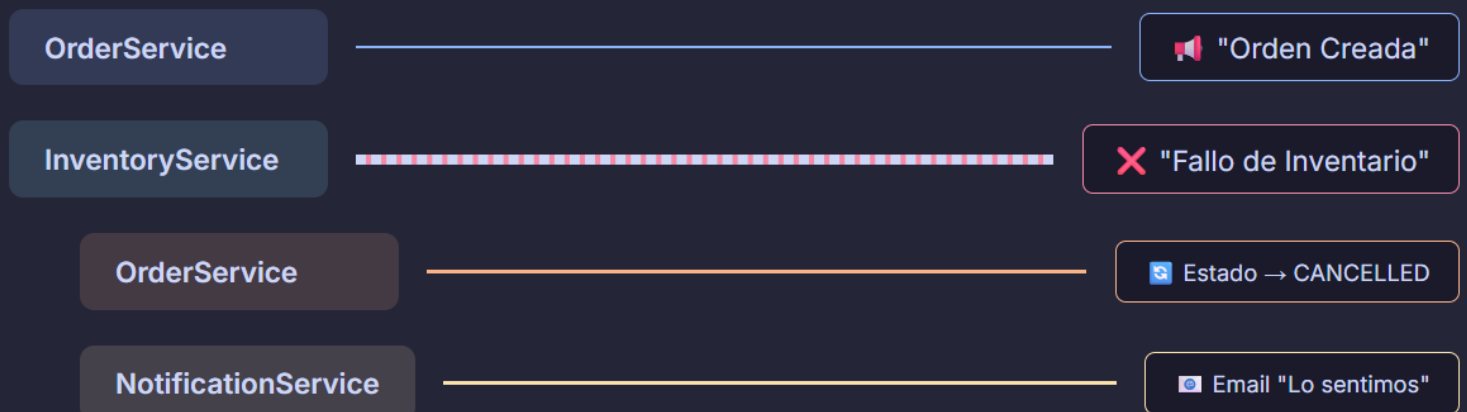
3 Paso 3: Compensación

Cuando escuchan "Fallo de Inventario":

- ▶ `OrderService` → Acción: Cambia estado a `CANCELLED` en su BD
- ▶ `NotificationService` → Acción: Envía email de "Lo sentimos"

Flujo Completo: Caso de Fallo

Cuando no hay stock disponible



Resultado: El sistema vuelve a un estado consistente. El cliente recibe una notificación clara del problema.

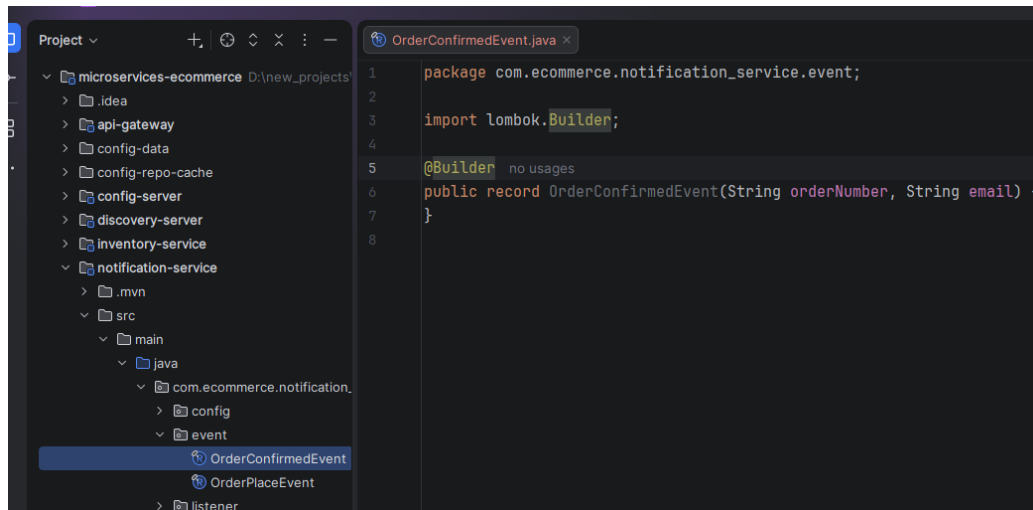
Saga Pattern en Resumen

- ✓ **Saga Pattern** resuelve transacciones distribuidas sin rollback mágico
- ✉ Usa **transacciones de compensación** para deshacer cambios cuando algo falla
- 👯 **Coreografía** (eventos) es ideal para sistemas desacoplados con RabbitMQ
- ⚠ Cada servicio debe **reaccionar a eventos de fallo** y compensar sus acciones

En microservicios, la consistencia eventual es aceptable. Lo importante es que el sistema siempre vuelva a un estado válido.

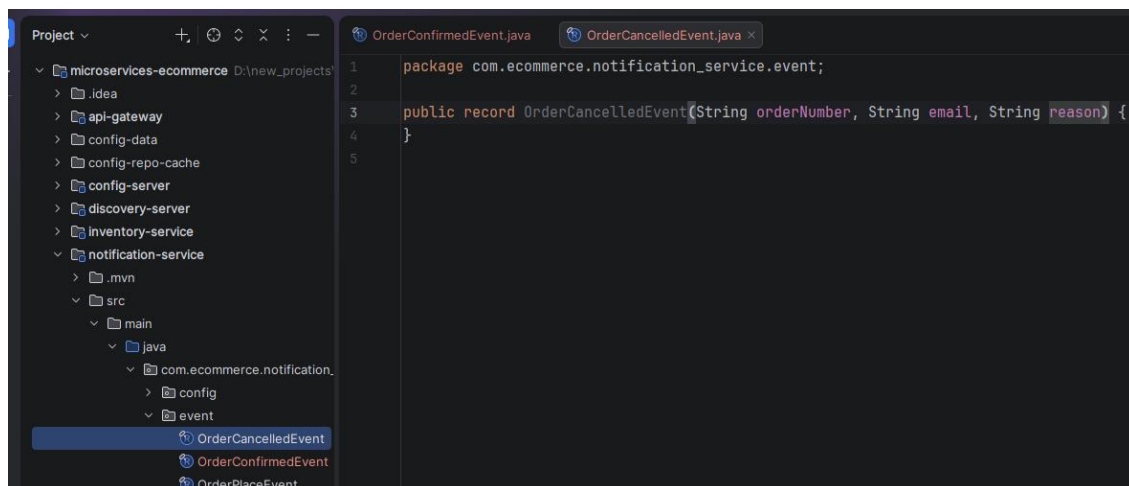
137. Order confirmada y Order cancelada

➔ En notification-service, crear un record para la data que será enviada al evento de confirmación y otro record de cancelación:



The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure is for 'microservices-ecommerce' and includes a 'notification-service' module. The code editor shows the 'OrderConfirmedEvent.java' file with the following content:

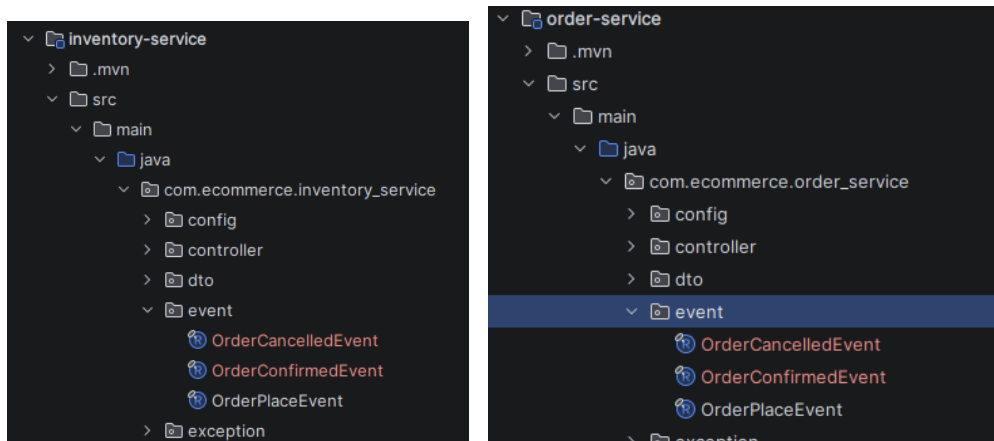
```
1 package com.ecommerce.notification_service.event;
2
3 import lombok.Builder;
4
5 @Builder no usages
6 public record OrderConfirmedEvent(String orderNumber, String email) {
7 }
8
```



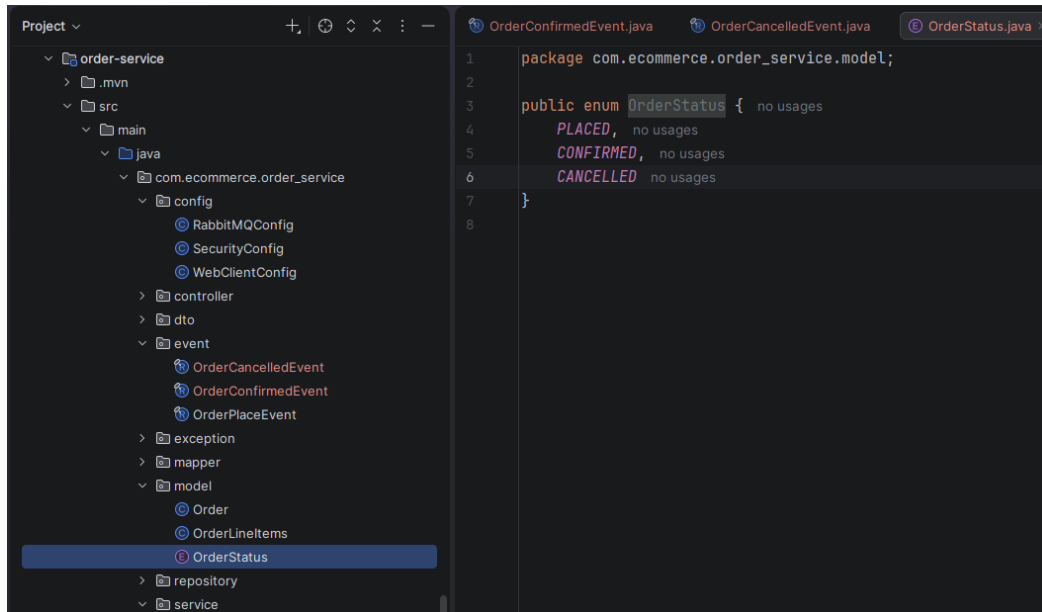
The screenshot shows the same IDE with the project structure on the left and a code editor on the right. The code editor shows the 'OrderCancelledEvent.java' file with the following content:

```
1 package com.ecommerce.notification_service.event;
2
3 public record OrderCancelledEvent(String orderNumber, String email, String reason) {
4 }
5
```

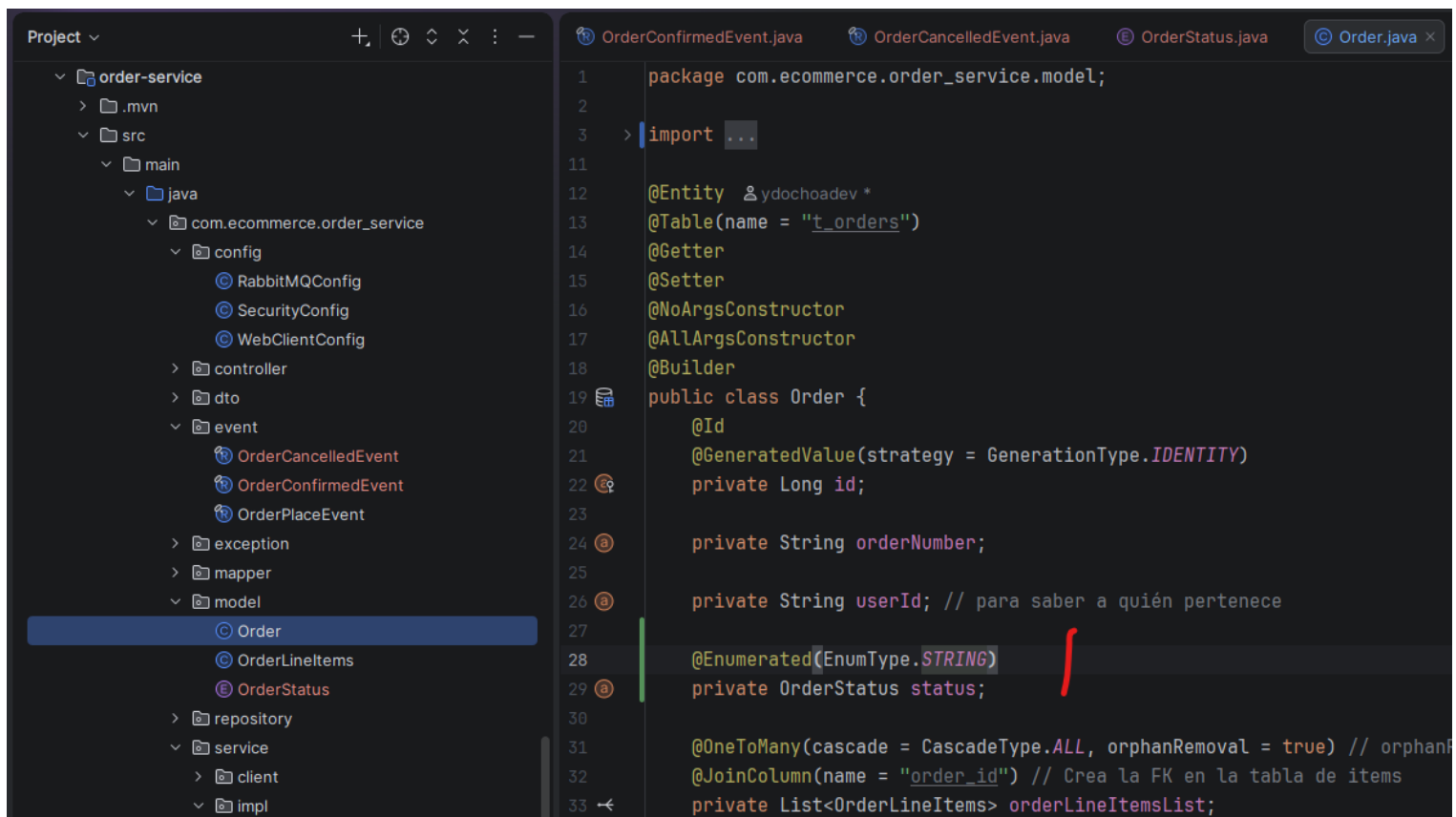
→ En inventory-service y order-service, copiar esos records:



→ Luego, en order-service, crear un enum como modelo para los estados:



→ Modificando la clase Order para agregar el estado en la BD. Y en el DTO de respuesta:



The screenshot shows an IDE with a project explorer on the left and a code editor on the right. The project explorer shows the structure of the 'order-service' project, with 'OrderResponse' selected under the 'dto' package. The code editor displays the 'OrderResponse.java' file, which is a Data Transfer Object (DTO) class. It includes package declarations, imports, annotations like '@Data', '@AllArgsConstructor', '@NoArgsConstructor', and '@Builder', and the class definition with private fields for 'id', 'orderNumber', 'status', and 'orderLineItemsDtoList'.

```
1 package com.ecommerce.order_service.dto;
2
3 import ...
4
5
6
7
8
9
10
11 @Data 19 usages ydochoadev *
12 @AllArgsConstructor
13 @NoArgsConstructor
14 @Builder
15 public class OrderResponse {
16     private Long id;
17     private String orderNumber;
18     private OrderStatus status;
19     private List<OrderLineItemsResponse> orderLineItemsDtoList;
20 }
21
```

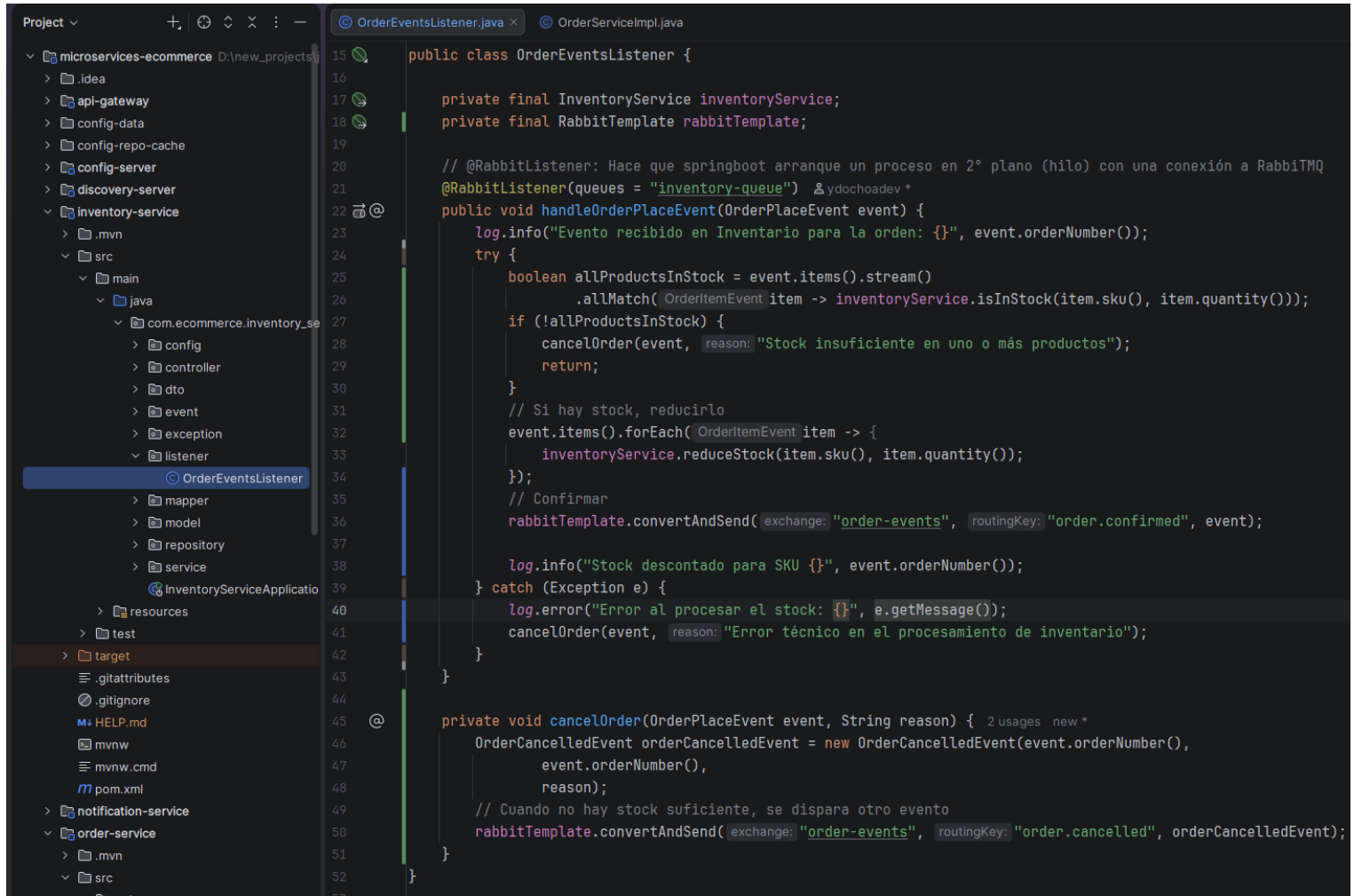
➔ En la implementación, cuando se hace el pedido, tendrá un primer estado de PLACED (orden pedida)

The screenshot shows the same IDE with the 'OrderServiceImpl' class selected in the project explorer. The code editor displays the implementation of the 'placeOrder' method. It includes a check for 'ordersEnabled', a warning and exception if disabled, a log message, a call to 'orderMapper.toOrder', setting the user ID, a loop to reduce stock for each item in the order, setting the order number and status to 'PLACED', saving the order, logging the success, and finally returning a list of 'OrderPlaceEvent' objects.

```
27 public class OrderServiceImpl implements OrderService {
28     // ...
29     @Retry(name = "inventory") */
30     public OrderResponse placeOrder(OrderRequest orderRequest, String userId) {
31         if (!ordersEnabled) {
32             log.warn("Pedido rechazado: Servicio deshabilitado por configuración");
33             throw new RuntimeException("El servicio de pedidos está en mantenimiento");
34         }
35
36         log.info("Colocando nueva orden...");
37         Order order = orderMapper.toOrder(orderRequest);
38         order.setUserId(userId);
39
40         /*for (var item : order.getOrderLineItemsList()) {
41             String sku = item.getSku();
42             Integer quantity = item.getQuantity();
43             // Realizar la llamada a inventory
44             try {
45                 inventoryClient.reduceStock(sku, quantity);
46             } catch (Exception e) {
47                 log.error("Error al reducir stock del producto {}. {}", sku, e.getMessage());
48                 throw new IllegalArgumentException("No se pudo procesa la orden: Stock");
49             }
50         }*/
51
52         order.setOrderNumber(UUID.randomUUID().toString());
53         // Estado del pedido
54         order.setStatus(OrderStatus.PLACED);
55         // Guardamos y capturamos la entidad persistida
56         Order savedOrder = orderRepository.save(order);
57         log.info("Orden guardada con éxito. ID: {}", savedOrder.getId());
58         // Contrato
59         List<OrderPlaceEvent.OrderItemEvent> orderItems = order.getOrderLineItemsList()
60     }
61 }

```

➔ Luego, en inventory-service, en el listener, ya que el inventario tiene que confirmar el stock cuando el evento se lanza desde order-service:



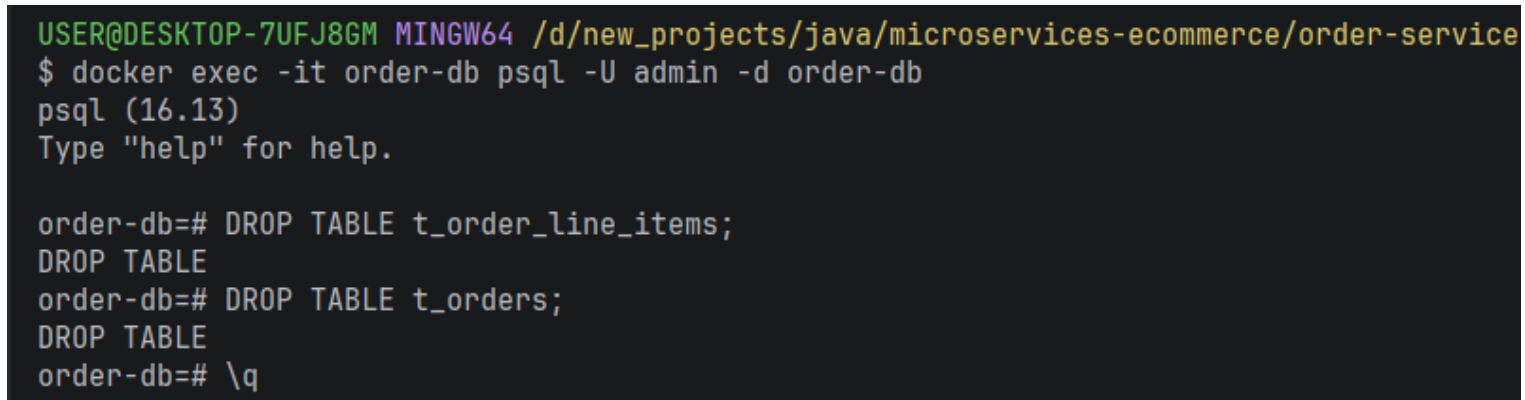
```
15 public class OrderEventsListener {
16
17     private final InventoryService inventoryService;
18     private final RabbitTemplate rabbitTemplate;
19
20     // @RabbitListener: Hace que springboot arranque un proceso en 2° plano (hilo) con una conexión a RabbitMQ
21     @RabbitListener(queues = "inventory-queue") & ydochoadev *
22     public void handleOrderPlaceEvent(OrderPlaceEvent event) {
23         log.info("Evento recibido en Inventario para la orden: {}", event.orderNumber());
24         try {
25             boolean allProductsInStock = event.items().stream()
26                 .allMatch( OrderItemEvent item -> inventoryService.isInStock(item.sku(), item.quantity()));
27             if (!allProductsInStock) {
28                 cancelOrder(event, reason: "Stock insuficiente en uno o más productos");
29                 return;
30             }
31             // Si hay stock, reducirlo
32             event.items().forEach( OrderItemEvent item -> {
33                 inventoryService.reduceStock(item.sku(), item.quantity());
34             });
35             // Confirmar
36             rabbitTemplate.convertAndSend( exchange: "order-events", routingKey: "order.confirmed", event);
37
38             log.info("Stock descontado para SKU {}", event.orderNumber());
39         } catch (Exception e) {
40             log.error("Error al procesar el stock: {}", e.getMessage());
41             cancelOrder(event, reason: "Error técnico en el procesamiento de inventario");
42         }
43     }
44
45     @
46     private void cancelOrder(OrderPlaceEvent event, String reason) { 2 usages new *
47         OrderCancelledEvent orderCancelledEvent = new OrderCancelledEvent(event.orderNumber(),
48             event.orderNumber(),
49             reason);
50         // Cuando no hay stock suficiente, se dispara otro evento
51         rabbitTemplate.convertAndSend( exchange: "order-events", routingKey: "order.cancelled", orderCancelledEvent);
52     }
53 }
```

138. Limpieza de persistencia vía Docker

➔ Como se agregaron algunos campos en la BD (enum de estado) en order-service:

Se debe ir a la consola de postgres mediante docker para eliminar la tabla:

\$ docker exec -it order-db psql -U admin -d order-db



```
USER@DESKTOP-7UFJ8GM MINGW64 /d/new_projects/java/microservices-ecommerce/order-service
$ docker exec -it order-db psql -U admin -d order-db
psql (16.13)
Type "help" for help.

order-db=# DROP TABLE t_order_line_items;
DROP TABLE
order-db=# DROP TABLE t_orders;
DROP TABLE
order-db=# \q
```

También:



```
USER@DESKTOP-7UFJ8GM MINGW64 /d/new_projects/java/microservices-ecommerce/order-service (main)
$ mvn clean install -Dmaven.test.skip=true
[INFO] Scanning for projects...
```

➔ Probar

139. Notificaciones basadas en certezas

- ➔Problema: El cliente recibe un correo de confirmación apenas hace el pedido, incluso si no hay stock.
- ➔Limpiar las colas de Rabbit:

<http://localhost:15672/>

OverviewConnectionsChannelsExchangesQueues and StreamsAdmin

Queues

▼ All queues (2)

Pagination

Page 1 ▼ of 1 - Filter: ☐ Regex ?

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	inventory-queue	classic	D Args	running	0	0	0				
/	notification-queue	classic	D Args	running	0	0	0				

► Add a new queue

HTTP APIDocumentationTutorialsNew releasesCommercial editionCommercial supportDiscussionsDiscordPluginsGi

Message rates last minute ?

Currently idle

Details

Features

arguments: x-queue-type: classic
durable: true
queue storage version: 2

State

Consumers 0
Consumer capacity ? 0

Policy

Operator policy

Effective policy definition

Messages ?

Message body bytes ?

Process memory ? 4

► Consumers (0)

► Bindings (2)

► Publish message

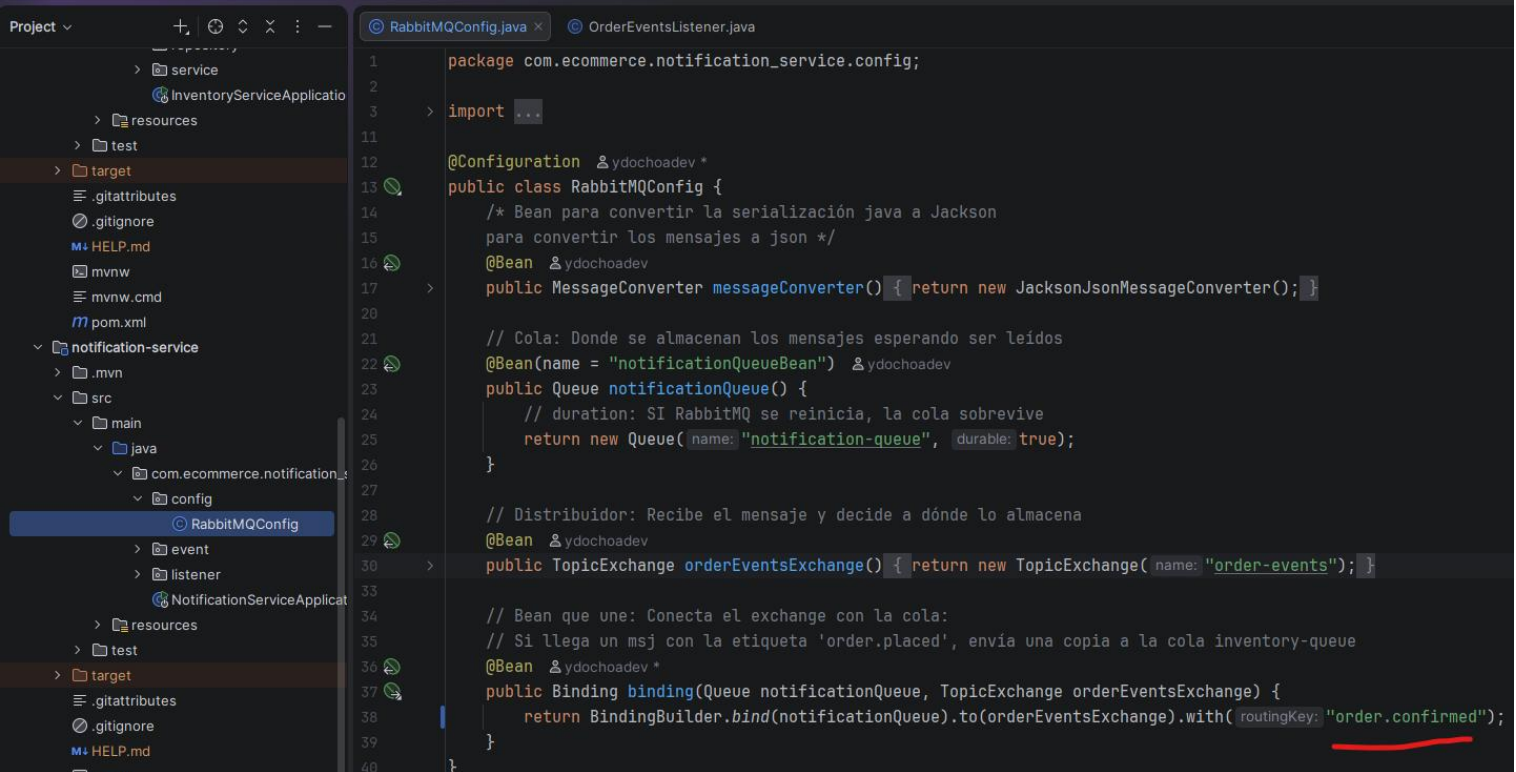
► Get messages

► Move messages

► Delete

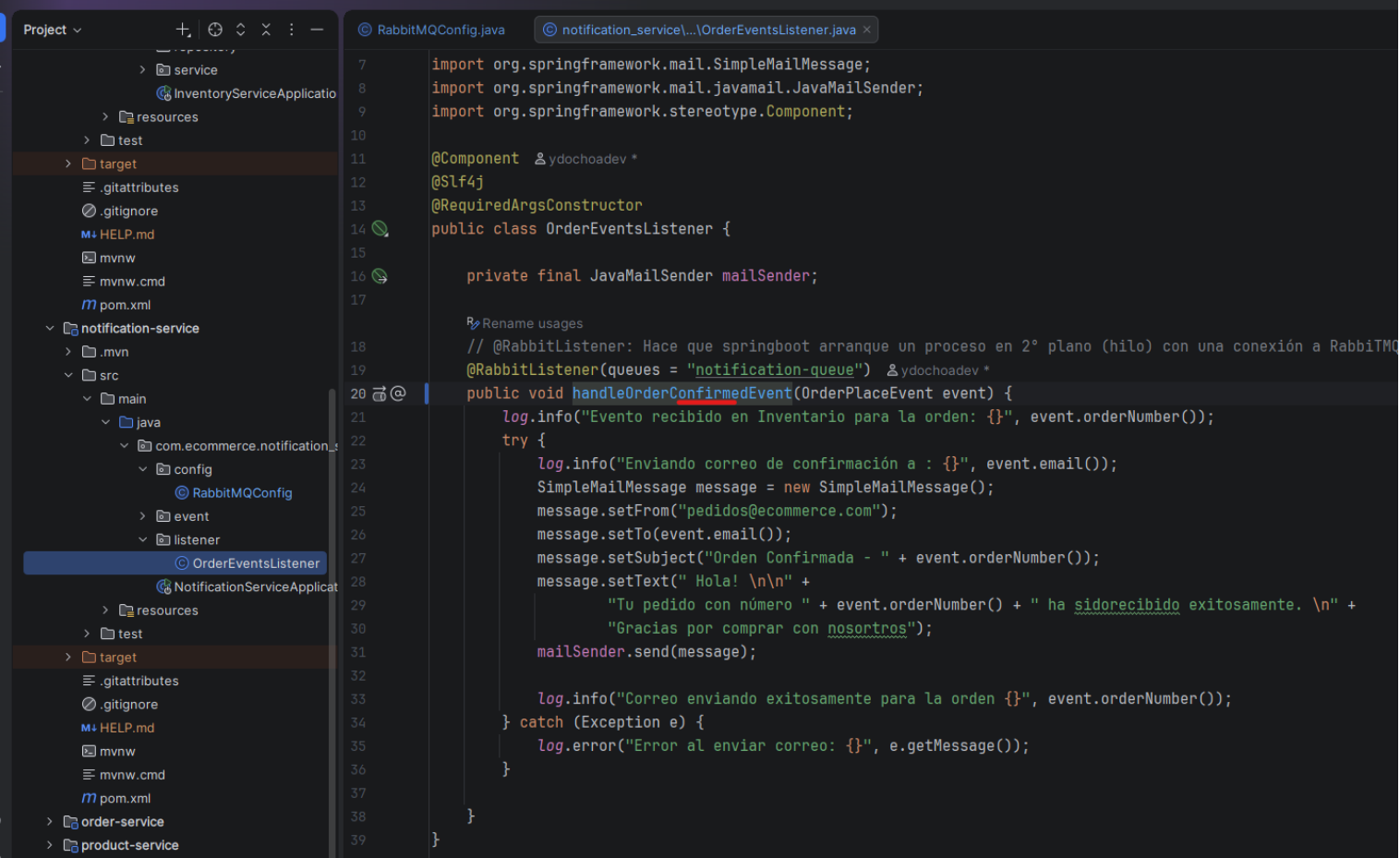
► Purge

** El pedido se debe confirmar cuando los productos del pedido tengan stock. Entonces, en notification-service (el que envía el correo al cliente) cambiar el routingKey a "order.confirmed"



```
1 package com.ecommerce.notification_service.config;
2
3 import ...
4
5 @Configuration &ydochoadev *
6 public class RabbitMQConfig {
7     /* Bean para convertir la serialización java a Jackson
8     para convertir los mensajes a json */
9     @Bean &ydochoadev
10     public MessageConverter messageConverter() { return new JacksonJsonMessageConverter(); }
11
12     // Cola: Donde se almacenan los mensajes esperando ser leídos
13     @Bean(name = "notificationQueueBean") &ydochoadev
14     public Queue notificationQueue() {
15         // duration: SI RabbitMQ se reinicia, la cola sobrevive
16         return new Queue(name: "notification-queue", durable: true);
17     }
18
19     // Distribuidor: Recibe el mensaje y decide a dónde lo almacena
20     @Bean &ydochoadev
21     public TopicExchange orderEventsExchange() { return new TopicExchange(name: "order-events"); }
22
23     // Bean que une: Conecta el exchange con la cola:
24     // Si llega un msj con la etiqueta 'order.placed', envía una copia a la cola inventory-queue
25     @Bean &ydochoadev *
26     public Binding binding(Queue notificationQueue, TopicExchange orderEventsExchange) {
27         return BindingBuilder.bind(notificationQueue).to(orderEventsExchange).with(routingKey: "order.confirmed");
28     }
29 }
```

En el listener:

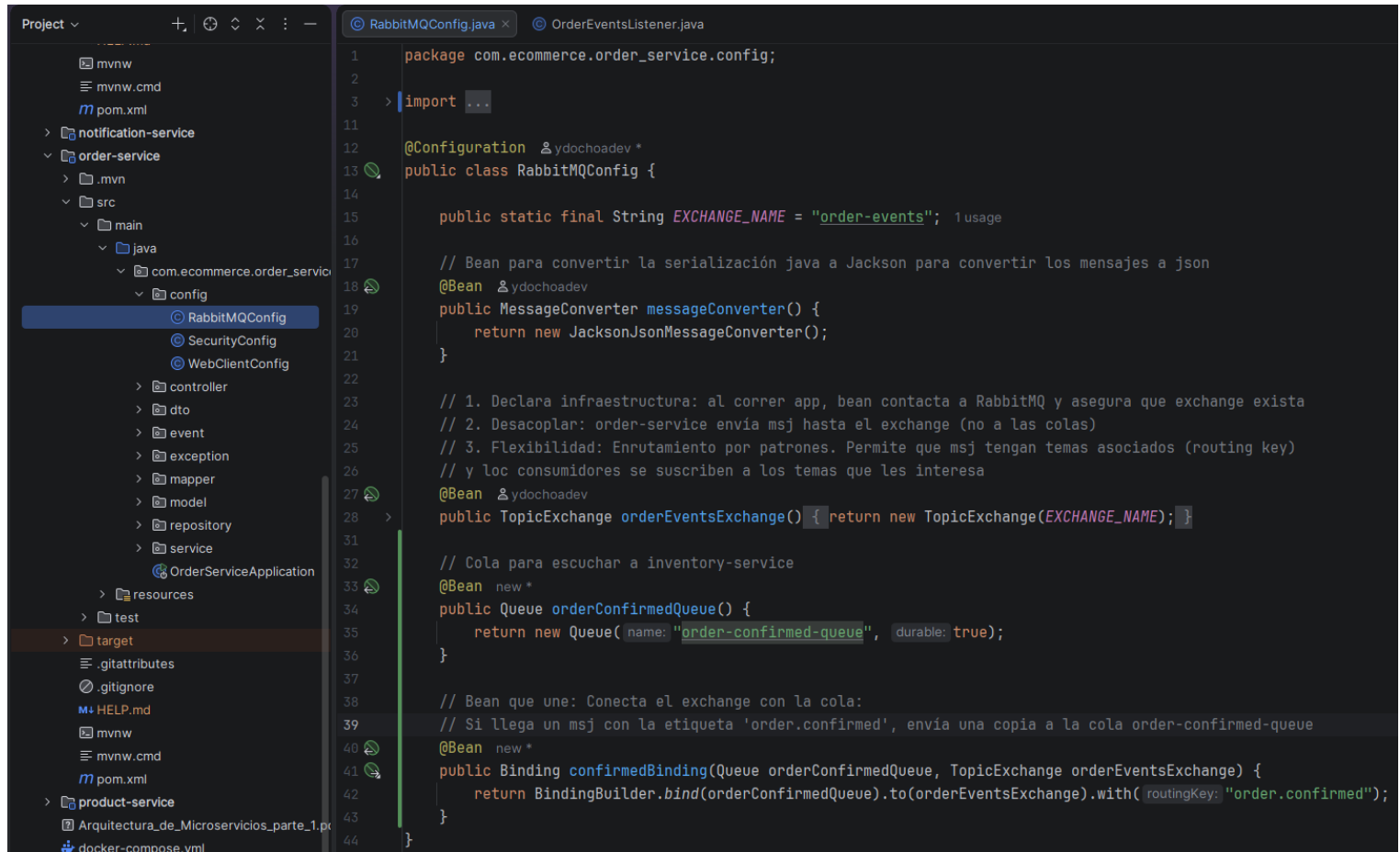


```
7 import org.springframework.mail.SimpleMailMessage;
8 import org.springframework.mail.javamail.JavaMailSender;
9 import org.springframework.stereotype.Component;
10
11 @Component &ydochoadev *
12 @Slf4j
13 @RequiredArgsConstructor
14 public class OrderEventsListener {
15
16     private final JavaMailSender mailSender;
17
18     // @RabbitListener: Hace que springboot arranque un proceso en 2º plano (hilo) con una conexión a RabbitMQ
19     @RabbitListener(queues = "notification-queue") &ydochoadev *
20     public void handleOrderConfirmedEvent(OrderPlaceEvent event) {
21         log.info("Evento recibido en Inventario para la orden: {}", event.orderNumber());
22         try {
23             log.info("Enviando correo de confirmación a : {}", event.email());
24             SimpleMailMessage message = new SimpleMailMessage();
25             message.setFrom("pedidos@ecommerce.com");
26             message.setTo(event.email());
27             message.setSubject("Orden Confirmada - " + event.orderNumber());
28             message.setText("Hola! \n\n" +
29                 "Tu pedido con número " + event.orderNumber() + " ha sido recibido exitosamente. \n" +
30                 "Gracias por comprar con nosotros");
31             mailSender.send(message);
32
33             log.info("Correo enviando exitosamente para la orden {}", event.orderNumber());
34         } catch (Exception e) {
35             log.error("Error al enviar correo: {}", e.getMessage());
36         }
37     }
38 }
```

** El método, con ese nombre, ata el listener con la configuración realizada anteriormente, con order.confirmed.

140. Consistencia eventual y actualización "status" parte 1

➔ Es necesario que el micro de pedidos tenga su propia cola para escuchar al micro de inventario.



```
1 package com.ecommerce.order.service.config;
2
3 > import ...
4
11
12 @Configuration & ydochoadev *
13 public class RabbitMQConfig {
14
15     public static final String EXCHANGE_NAME = "order-events"; 1 usage
16
17     // Bean para convertir la serialización java a Jackson para convertir los mensajes a json
18     @Bean & ydochoadev
19     public MessageConverter messageConverter() {
20         return new JacksonJsonMessageConverter();
21     }
22
23     // 1. Declara infraestructura: al correr app, bean contacta a RabbitMQ y asegura que exchange exista
24     // 2. Desacoplar: order-service envía msj hasta el exchange (no a las colas)
25     // 3. Flexibilidad: Enrutamiento por patrones. Permite que msj tengan temas asociados (routing key)
26     // y los consumidores se suscriben a los temas que les interesa
27     @Bean & ydochoadev
28     public TopicExchange orderEventsExchange() { return new TopicExchange(EXCHANGE_NAME); }
29
30     // Cola para escuchar a inventory-service
31     @Bean new *
32     public Queue orderConfirmedQueue() {
33         return new Queue( name: "order-confirmed-queue", durable: true);
34     }
35
36     // Bean que une: Conecta el exchange con la cola:
37     // Si llega un msj con la etiqueta 'order.confirmed', envía una copia a la cola order-confirmed-queue
38     @Bean new *
39     public Binding confirmedBinding(Queue orderConfirmedQueue, TopicExchange orderEventsExchange) {
40         return BindingBuilder.bind(orderConfirmedQueue).to(orderEventsExchange).with( routingKey: "order.confirmed");
41     }
42
43
44 }
```

Los parámetros de entrada en el método confirmedBinding deben tener el mismo nombre de los beans (de la cola y el exchange), para que spring haga la conexión automática.

➔ Crear otra cola para las órdenes canceladas:

```
// Cola para escuchar a inventory-service de las órdenes canceladas
@Bean new *
public Queue orderCancelledQueue() {
    return new Queue( name: "order-cancelled-queue", durable: true);
}

// Bean que une: Conecta el exchange con la cola:
// Si llega un msj con la etiqueta 'order.confirmed', envía una copia a la cola order-cancelled-queue
@Bean new *
public Binding cancelledBinding(Queue orderCancelledQueue, TopicExchange orderEventsExchange) {
    return BindingBuilder.bind(orderCancelledQueue).to(orderEventsExchange).with( routingKey: "order.cancelled");
}
```


Quedaría:

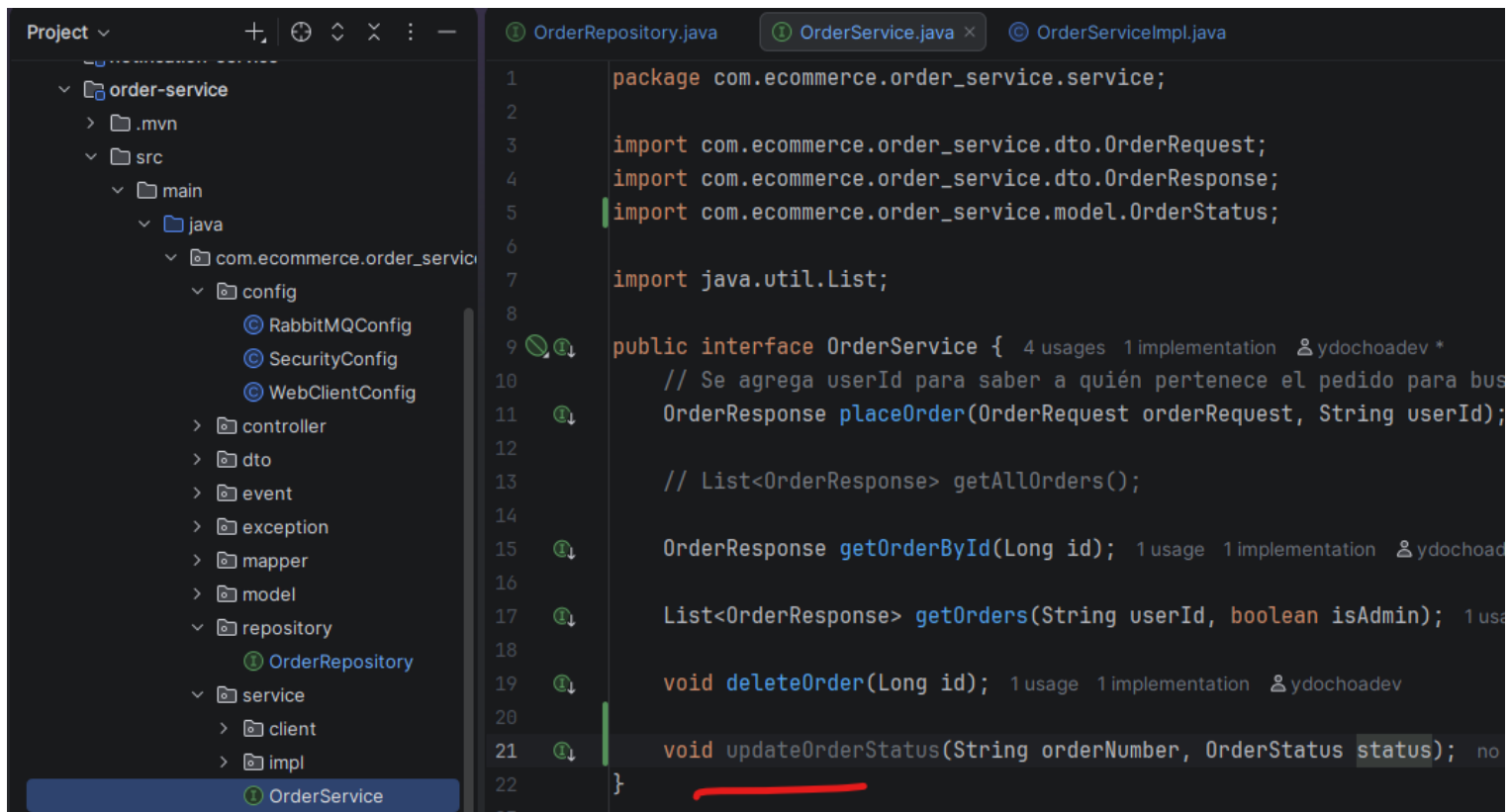
```
RabbitMQConfig.java x OrderEventListener.java
13 public class RabbitMQConfig {
18 @Bean @ydochoadev
19 public MessageConverter messageConverter() {
20     return new JacksonJsonMessageConverter();
21 }
22
23 // 1. Declara infraestructura: al correr app, bean contacta a RabbitMQ y asegura que exchange exista
24 // 2. Desacoplar: order-service envía msj hasta el exchange (no a las colas)
25 // 3. Flexibilidad: Enrutamiento por patrones. Permite que msj tengan temas asociados (routing key)
26 // y los consumidores se suscriben a los temas que les interesa
27 @Bean @ydochoadev
28 public TopicExchange orderEventsExchange() { return new TopicExchange(EXCHANGE_NAME); }
31
32 // Cola para escuchar a inventory-service de las órdenes confirmadas
33 @Bean new *
34 public Queue orderConfirmedQueue() {
35     return new Queue(name: "order-confirmed-queue", durable: true);
36 }
37
38 // Bean que une: Conecta el exchange con la cola:
39 // Si llega un msj con la etiqueta 'order.confirmed', envía una copia a la cola order-confirmed-queue
40 @Bean new *
41 public Binding confirmedBinding(Queue orderConfirmedQueue, TopicExchange orderEventsExchange) {
42     return BindingBuilder.bind(orderConfirmedQueue).to(orderEventsExchange).with(routingKey: "order.confirmed");
43 }
44
45 // Cola para escuchar a inventory-service de las órdenes canceladas
46 @Bean new *
47 public Queue orderCancelledQueue() {
48     return new Queue(name: "order-cancelled-queue", durable: true);
49 }
50
51 // Bean que une: Conecta el exchange con la cola:
52 // Si llega un msj con la etiqueta 'order.confirmed', envía una copia a la cola order-cancelled-queue
53 @Bean new *
54 public Binding cancelledBinding(Queue orderCancelledQueue, TopicExchange orderEventsExchange) {
55     return BindingBuilder.bind(orderCancelledQueue).to(orderEventsExchange).with(routingKey: "order.cancelled");
56 }
57 }
```

➔Repository, agregar función para buscar por orderNumber:

```
Project
└─ order-service
   └─ src
      └─ main
         └─ java
            └─ com.ecommerce.order_service
               └─ config
                  └─ RabbitMQConfig
                  └─ SecurityConfig
                  └─ WebClientConfig
               └─ controller
               └─ dto
               └─ event
               └─ exception
               └─ mapper
               └─ model
               └─ repository
                  └─ OrderRepository
               └─ service
```

```
OrderRepository.java x
1 package com.ecommerce.order_service.repository;
2
3 import ...
4
5
6
7
8
9 public interface OrderRepository extends JpaRepository<Order, Long> {
10     // Búsqueda por userId
11     List<Order> findById(String userId); 1 usage @ydochoadev
12
13     Optional<Order> findByOrderNumber(String orderNumber); no usages new
14 }
15
```

➔ Actualizar status

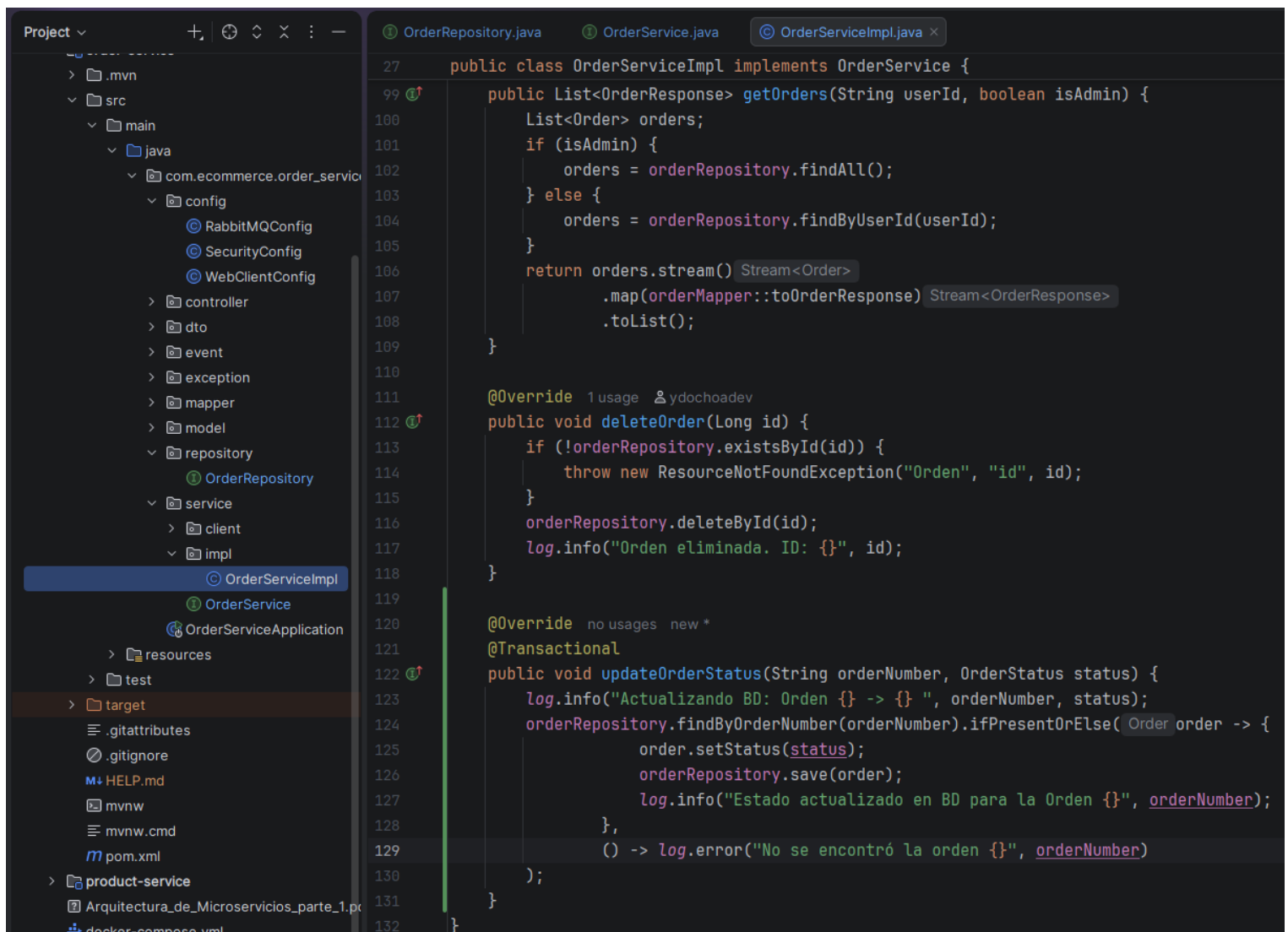


```
Project ▾
├── .mvn
├── src
│   ├── main
│   │   └── java
│   │       ├── com.ecommerce.order_service
│   │       │   ├── config
│   │       │   │   ├── RabbitMQConfig
│   │       │   │   ├── SecurityConfig
│   │       │   │   └── WebClientConfig
│   │       │   ├── controller
│   │       │   ├── dto
│   │       │   ├── event
│   │       │   ├── exception
│   │       │   ├── mapper
│   │       │   ├── model
│   │       │   ├── repository
│   │       │   │   ├── OrderRepository
│   │       │   └── service
│   │       │       ├── client
│   │       │       └── impl
│   │       └── OrderService
└── target

OrderRepository.java  OrderService.java  OrderServiceImpl.java

1  package com.ecommerce.order_service.service;
2
3  import com.ecommerce.order_service.dto.OrderRequest;
4  import com.ecommerce.order_service.dto.OrderResponse;
5  import com.ecommerce.order_service.model.OrderStatus;
6
7  import java.util.List;
8
9  public interface OrderService { 4 usages 1 implementation ydochoadev *
10     // Se agrega userId para saber a quién pertenece el pedido para bus
11     OrderResponse placeOrder(OrderRequest orderRequest, String userId);
12
13     // List<OrderResponse> getAllOrders();
14
15     OrderResponse getOrderById(Long id); 1 usage 1 implementation ydochoadev
16
17     List<OrderResponse> getOrders(String userId, boolean isAdmin); 1 usa
18
19     void deleteOrder(Long id); 1 usage 1 implementation ydochoadev
20
21     void updateOrderStatus(String orderNumber, OrderStatus status); no
22 }
23
```

Implementar:



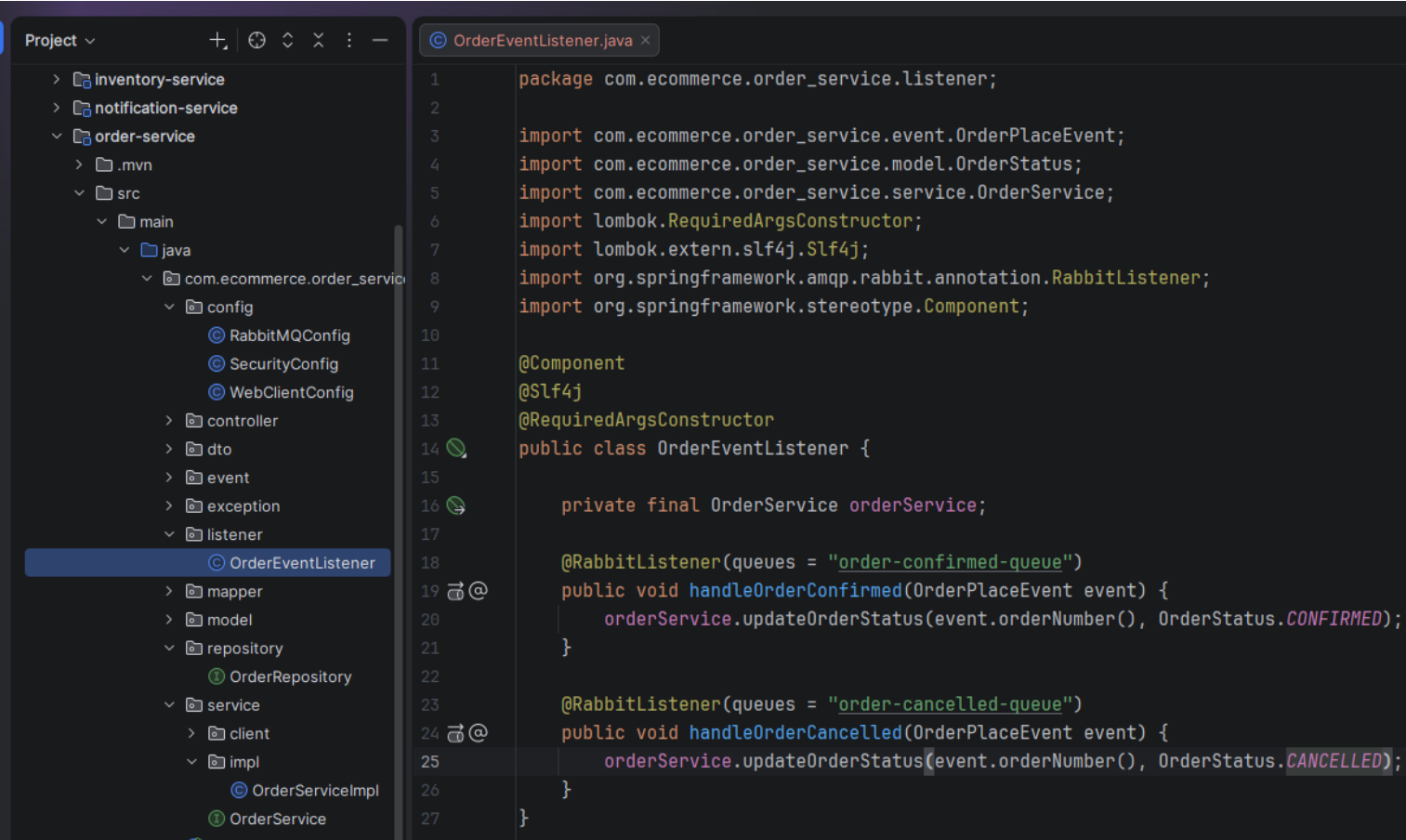
```
Project ▾
├── .mvn
├── src
│   ├── main
│   │   └── java
│   │       ├── com.ecommerce.order_service
│   │       │   ├── config
│   │       │   │   ├── RabbitMQConfig
│   │       │   │   ├── SecurityConfig
│   │       │   │   └── WebClientConfig
│   │       │   ├── controller
│   │       │   ├── dto
│   │       │   ├── event
│   │       │   ├── exception
│   │       │   ├── mapper
│   │       │   ├── model
│   │       │   ├── repository
│   │       │   │   ├── OrderRepository
│   │       │   └── service
│   │       │       ├── client
│   │       │       └── impl
│   │       └── OrderService
└── target

OrderRepository.java  OrderService.java  OrderServiceImpl.java

27  public class OrderServiceImpl implements OrderService {
99      public List<OrderResponse> getOrders(String userId, boolean isAdmin) {
100          List<Order> orders;
101          if (isAdmin) {
102              orders = orderRepository.findAll();
103          } else {
104              orders = orderRepository.findById(userId);
105          }
106          return orders.stream().map(orderMapper::toOrderResponse).toList();
107      }
108
109      @Override 1 usage ydochoadev
110      public void deleteOrder(Long id) {
111          if (!orderRepository.existsById(id)) {
112              throw new ResourceNotFoundException("Orden", "id", id);
113          }
114          orderRepository.deleteById(id);
115          log.info("Orden eliminada. ID: {}", id);
116      }
117
118      @Override no usages new *
119      @Transactional
120      public void updateOrderStatus(String orderNumber, OrderStatus status) {
121          log.info("Actualizando BD: Orden {} -> {} ", orderNumber, status);
122          orderRepository.findById(orderNumber).ifPresentOrElse(
123              order -> {
124                  order.setStatus(status);
125                  orderRepository.save(order);
126                  log.info("Estado actualizado en BD para la Orden {}", orderNumber);
127              },
128              () -> log.error("No se encontró la orden {}", orderNumber)
129          );
130      }
131  }
132
```

141. Consistencia eventual y actualización "status" parte 2

➔ Agregar un paquete listener para order-service:

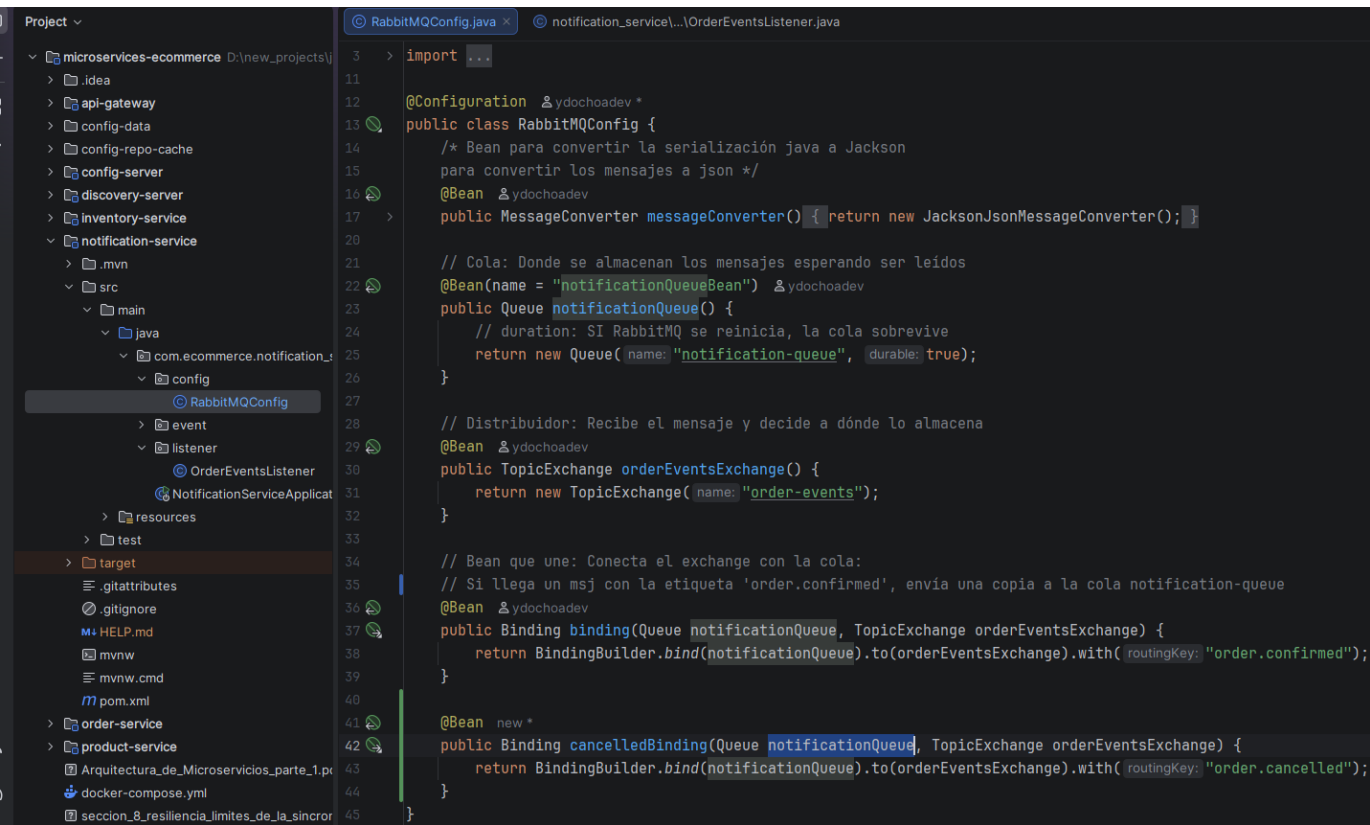


→ Probar.

142. Patrón Saga: método 1 - Consumo competitivo

➔ Cliente debe saber por qué el pedido fue cancelado.

➔ En notification-service, en el listener y configuración de Rabbit:

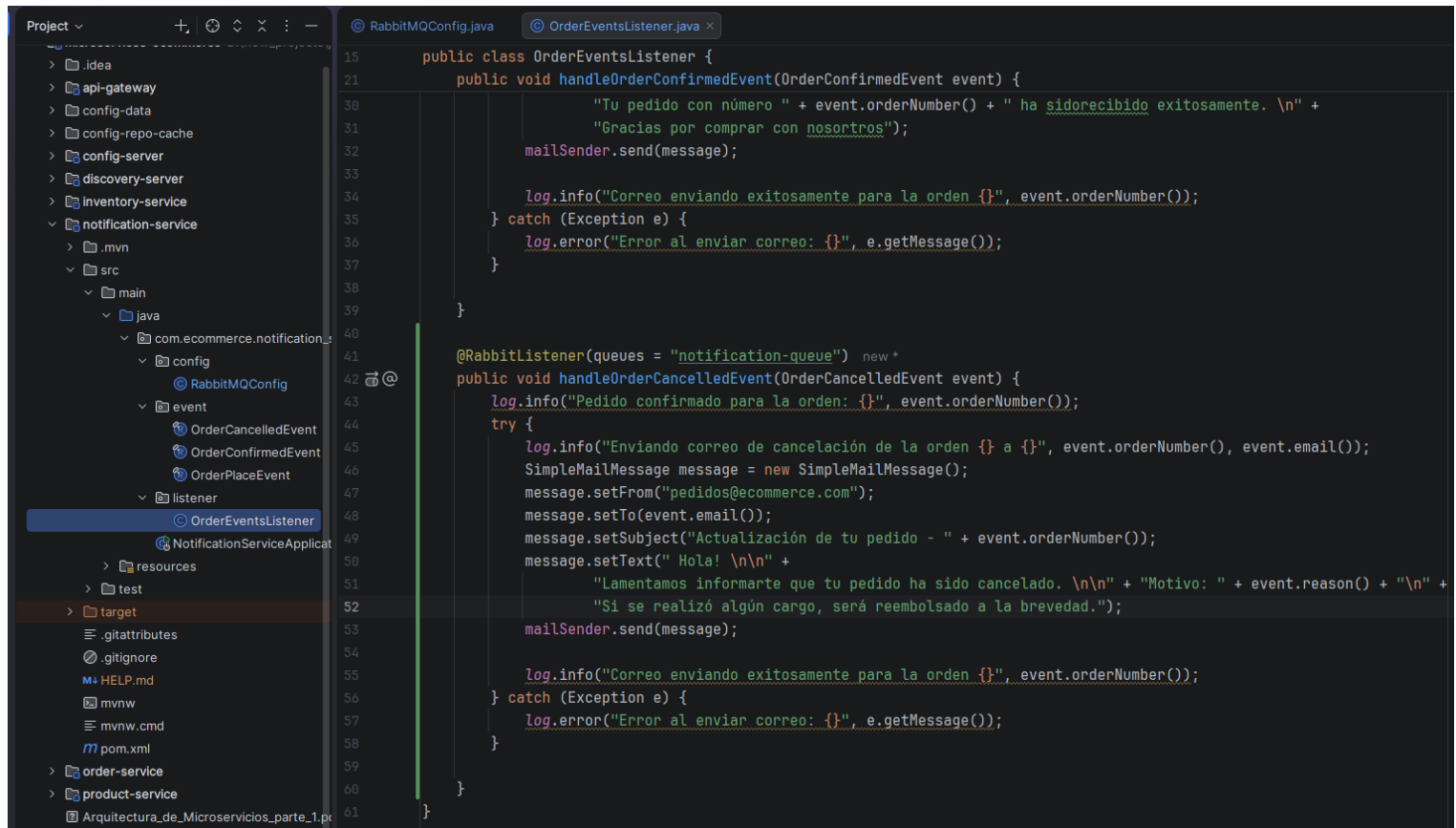


→ En el listener:

Los nombres de los métodos son importantes para que spring sepa a qué evento escuchar (debe tener la palabra del “routingKey”):

handleOrder**Confirmed**Event => Escucha a las órdenes confirmadas.

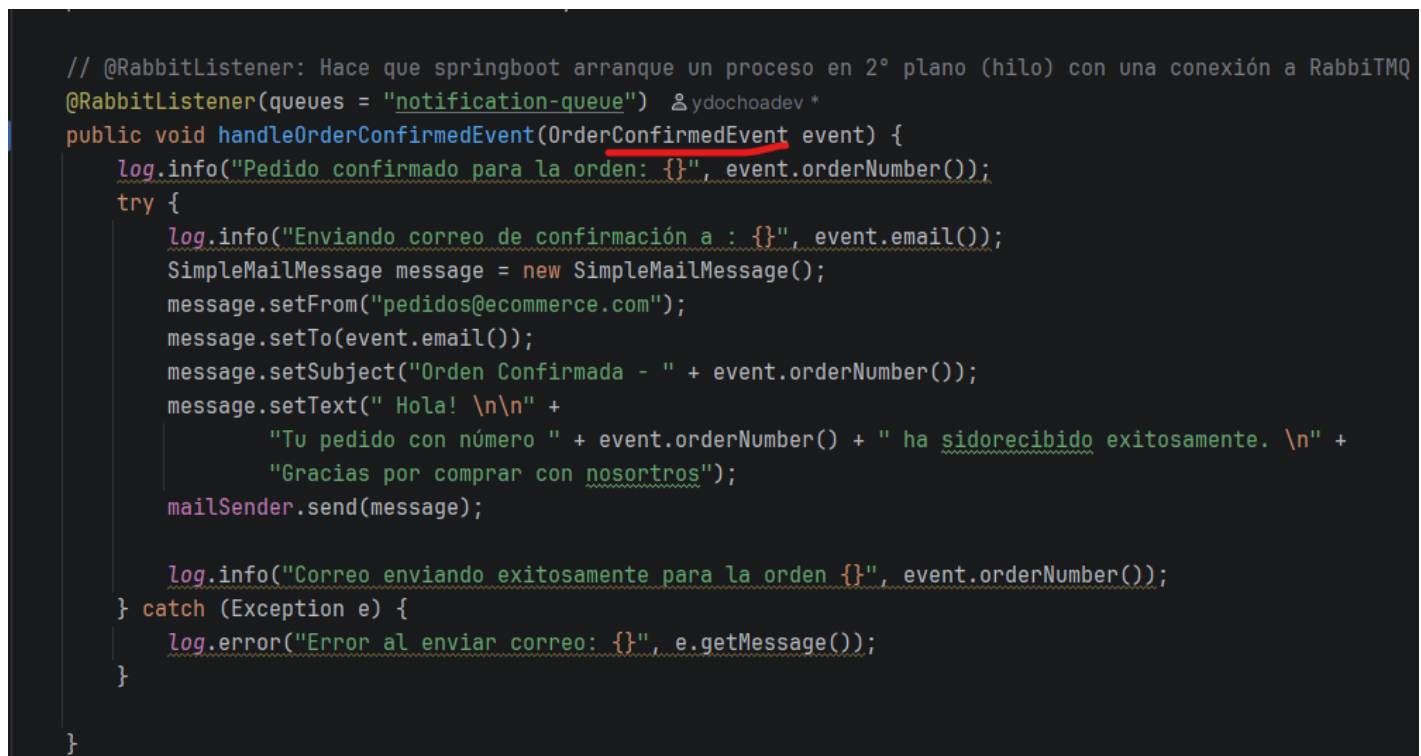
handleOrder**Cancelled**Event => Escucha a las órdenes canceladas.



The screenshot shows an IDE with a project structure on the left and the `OrderEventsListener.java` file open in the editor. The project structure includes a `notification-service` module with a `listener` package containing `OrderEventsListener`. The code in the editor defines a `OrderEventsListener` class with two methods: `handleOrderConfirmedEvent` and `handleOrderCancelledEvent`. Both methods use `log` for logging, `mailSender` for sending emails, and `SimpleMailMessage` for creating email messages. The `handleOrderCancelledEvent` method also includes a `try` block to handle exceptions.

```
15 public class OrderEventsListener {
21     public void handleOrderConfirmedEvent(OrderConfirmedEvent event) {
30         "Tu pedido con número " + event.orderNumber() + " ha sido recibido exitosamente. \n" +
31         "Gracias por comprar con nosotros");
32         mailSender.send(message);
33
34         log.info("Correo enviando exitosamente para la orden {}", event.orderNumber());
35     } catch (Exception e) {
36         log.error("Error al enviar correo: {}", e.getMessage());
37     }
38
39 }
40
41 @RabbitListener(queues = "notification-queue") new *
42 public void handleOrderCancelledEvent(OrderCancelledEvent event) {
43     log.info("Pedido confirmado para la orden: {}", event.orderNumber());
44     try {
45         log.info("Enviando correo de cancelación de la orden {} a {}", event.orderNumber(), event.email());
46         SimpleMailMessage message = new SimpleMailMessage();
47         message.setFrom("pedidos@ecommerce.com");
48         message.setTo(event.email());
49         message.setSubject("Actualización de tu pedido - " + event.orderNumber());
50         message.setText(" Hola! \n\n" +
51             "Lamentamos informarte que tu pedido ha sido cancelado. \n\n" + "Motivo: " + event.reason() + "\n" +
52             "Si se realizó algún cargo, será reembolsado a la brevedad.");
53         mailSender.send(message);
54
55         log.info("Correo enviando exitosamente para la orden {}", event.orderNumber());
56     } catch (Exception e) {
57         log.error("Error al enviar correo: {}", e.getMessage());
58     }
59 }
60
61 }
```

Para una mejor lectura:



This screenshot shows the same `OrderEventsListener.java` file with syntax highlighting. The `handleOrderConfirmedEvent` method is highlighted in red, and the `handleOrderCancelledEvent` method is highlighted in blue. The code is as follows:

```
// @RabbitListener: Hace que springboot arranque un proceso en 2º plano (hilo) con una conexión a RabbitMQ
// @RabbitListener(queues = "notification-queue") ydochoadev *
public void handleOrderConfirmedEvent(OrderConfirmedEvent event) {
    log.info("Pedido confirmado para la orden: {}", event.orderNumber());
    try {
        log.info("Enviando correo de confirmación a : {}", event.email());
        SimpleMailMessage message = new SimpleMailMessage();
        message.setFrom("pedidos@ecommerce.com");
        message.setTo(event.email());
        message.setSubject("Orden Confirmada - " + event.orderNumber());
        message.setText(" Hola! \n\n" +
            "Tu pedido con número " + event.orderNumber() + " ha sido recibido exitosamente. \n" +
            "Gracias por comprar con nosotros");
        mailSender.send(message);

        log.info("Correo enviando exitosamente para la orden {}", event.orderNumber());
    } catch (Exception e) {
        log.error("Error al enviar correo: {}", e.getMessage());
    }
}

}
```

143. Retry pattern: Reintentos automáticos

<https://gist.github.com/gchaldou/c11615ec71a526ae513ae1cf62914f9a>

→ en la configuración de notification-service, config-data:

```
1  eureka:
2    client:
3      service-url:
4        defaultZone: http://localhost:8761/eureka/
5
6    server:
7      port: 0
8
9    logging:
10     level:
11       root: INFO
12       com.ecommerce: INFO
13
14 # --- NUEVA CONFIGURACIÓN RETRY Y SMTP (Mailtrap) ---
15 spring:
16   rabbitmq:
17     listener:
18       simple:
19         retry:
20           enabled: true
21           initial-interval: 3000ms # Primera espera de 3 segundos
22           max-attempts: 3         # Máximo 3 intentos totales
23           max-interval: 10000ms  # No esperar más de 10 segundos
24           multiplier: 2.0       # Backoff exponencial (3s, 6s, 10s)
25
26   mail:
27     host: sandbox.smtp.mailtrap.io
28     port: 2525
29     username: 20622f2711ab82 # <--- Pega tu usuario aquí
30     password: 67fb9ab9904333 # <--- Pega tu password aquí
31     properties:
32       mail:
33         smtp:
34           auth: true
35           starttls:
36             enable: true
```

→ COMMITTEAR

→ Agregar dependencia de retry en el pom;

```
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/
33  <dependencies>
48    <groupId>org.springframework.cloud</groupId>
49    <artifactId>spring-cloud-starter-config</artifactId>
50  </dependency>
51  <!-- Agregando dependencia eureka-client -->
52  <dependency>
53    <groupId>org.springframework.cloud</groupId>
54    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
55  </dependency>
56  <!-- Dependencia para los reintentos -->
57  <dependency>
58    <groupId>org.springframework.retry</groupId>
59    <artifactId>spring-retry</artifactId>
60  </dependency>
61  <dependency>
62    <groupId>org.projectlombok</groupId>
63    <artifactId>lombok</artifactId>
64    <optional>true</optional>
65  </dependency>
66  <dependency>
67    <groupId>org.springframework.boot</groupId>
68    <artifactId>spring-boot-starter-amqp</artifactId>
69  </dependency>
```


➔ En el listener de notification-service, quitar los try-catch de los métodos; esto debido a que el catch, al haber un error, hace que todo termine bien, que no colapse la aplicación, y no llega a realizar los reintentos.

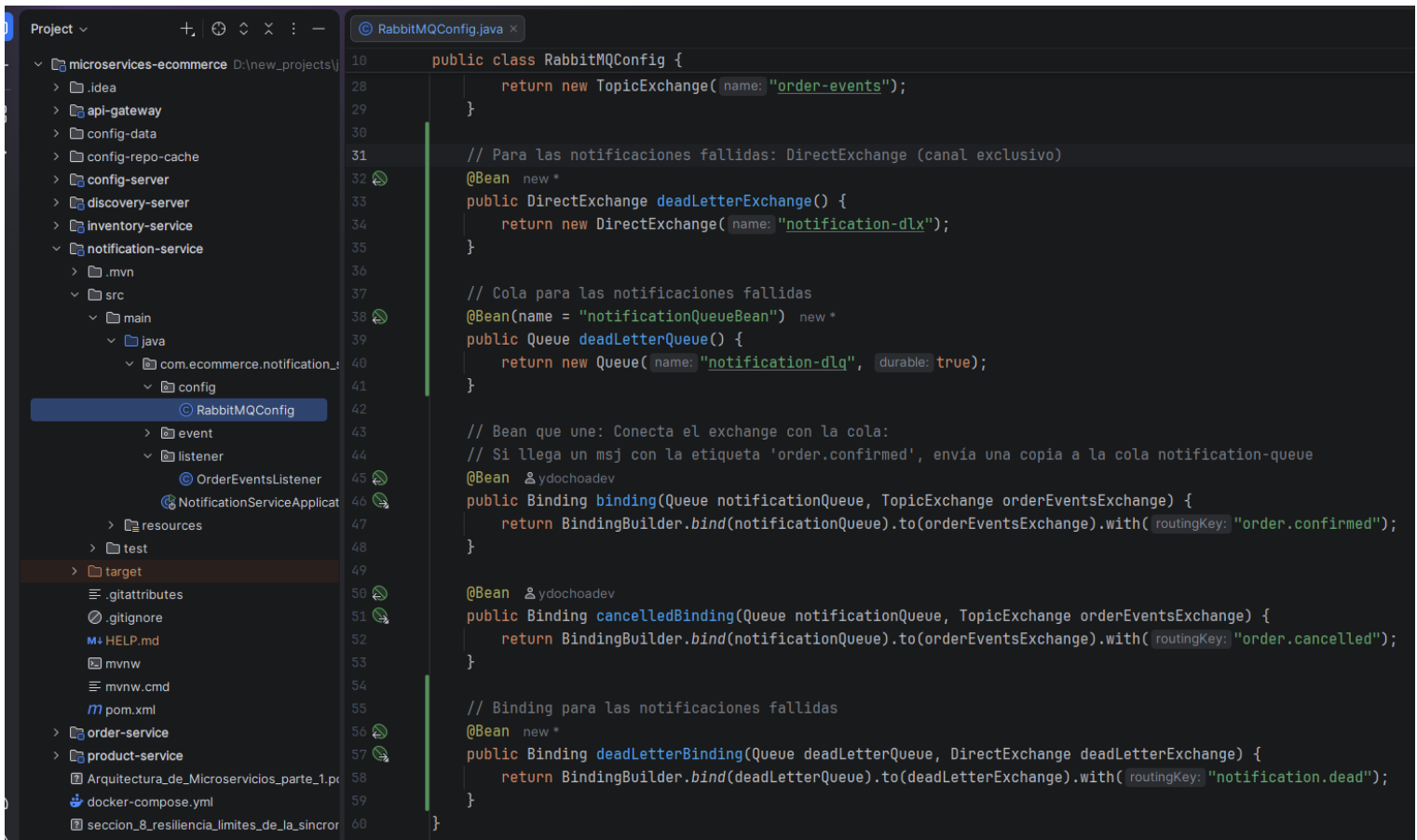
```
OrderEventsListener.java x
15 public class OrderEventsListener {
17     private final JavaMailSender mailSender;
18
19     // @RabbitListener: Hace que springboot arranque un proceso en 2° plano (hilo) con una conexión a RabbitMQ
20     @RabbitListener(queues = "notification-queue") ydochoadev
21     public void handleOrderConfirmedEvent(OrderConfirmedEvent event) {
22         log.info("Pedido confirmado para la orden: {}", event.orderNumber());
23         log.info("Enviando correo de confirmación a : {}", event.email());
24         SimpleMailMessage message = new SimpleMailMessage();
25         message.setFrom("pedidos@ecommerce.com");
26         message.setTo(event.email());
27         message.setSubject("Orden Confirmada - " + event.orderNumber());
28         message.setText(" Hola! \n\n" +
29             "Tu pedido con número " + event.orderNumber() + " ha sido recibido exitosamente. \n" +
30             "Gracias por comprar con nosotros");
31         mailSender.send(message);
32
33         log.info("Correo enviando exitosamente para la orden {}", event.orderNumber());
34     }
35
36     @RabbitListener(queues = "notification-queue") ydochoadev
37     public void handleOrderCancelledEvent(OrderCancelledEvent event) {
38         log.info("Pedido confirmado para la orden: {}", event.orderNumber());
39
40         log.info("Enviando correo de cancelación de la orden {} a {}", event.orderNumber(), event.email());
41         SimpleMailMessage message = new SimpleMailMessage();
42         message.setFrom("pedidos@ecommerce.com");
43         message.setTo(event.email());
44         message.setSubject("Actualización de tu pedido - " + event.orderNumber());
45         message.setText(" Hola! \n\n" +
46             "Lamentamos informarte que tu pedido ha sido cancelado. \n\n" + "Motivo: " + event.reason() + "\n" +
47             "Si se realizó algún cargo, será reembolsado a la brevedad.");
48         mailSender.send(message);
49
50         log.info("Correo enviando exitosamente para la orden {}", event.orderNumber());
51     }
52 }
```

144. Topic Exchange y Direct Exchange

➔ DLQ: Dead Letter Quee. Gestión de mensajes fallidos. Como un buzón de mensajes que no fueron entregados. Si el servicio de notificaciones no puede enviar las notificaciones luego de 3 intentos fallidos, RabbitMQ mueve ese mensaje a una cola especial para revisarla en otro momento, auditarla o reprocesarla.

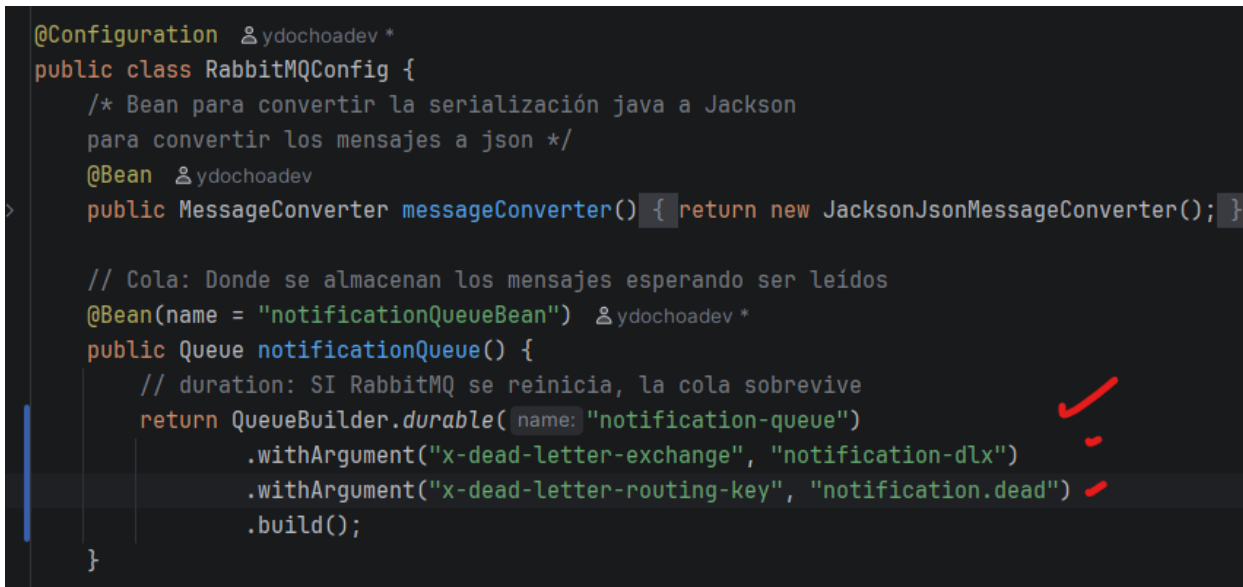
Direct Exchange es un canal exclusivo Topic Exchange: Con Topic Exchange puede haber un routingKey de esta manera "order.*" en donde se le puede enviar un * cuando se quiera que todas las notificaciones de órdenes sean gestionadas por una sola cola.

→ En notification-service:



```
10 public class RabbitMQConfig {
28     return new TopicExchange( name: "order-events");
29 }
30
31 // Para las notificaciones fallidas: DirectExchange (canal exclusivo)
32 @Bean new *
33 public DirectExchange deadLetterExchange() {
34     return new DirectExchange( name: "notification-dlx");
35 }
36
37 // Cola para las notificaciones fallidas
38 @Bean(name = "notificationQueueBean") new *
39 public Queue deadLetterQueue() {
40     return new Queue( name: "notification-dlq", durable: true);
41 }
42
43 // Bean que une: Conecta el exchange con la cola:
44 // Si llega un msj con la etiqueta 'order.confirmed', envía una copia a la cola notification-queue
45 @Bean @ydochoadev
46 public Binding binding(Queue notificationQueue, TopicExchange orderEventsExchange) {
47     return BindingBuilder.bind(notificationQueue).to(orderEventsExchange).with( routingKey: "order.confirmed");
48 }
49
50 @Bean @ydochoadev
51 public Binding cancelledBinding(Queue notificationQueue, TopicExchange orderEventsExchange) {
52     return BindingBuilder.bind(notificationQueue).to(orderEventsExchange).with( routingKey: "order.cancelled");
53 }
54
55 // Binding para las notificaciones fallidas
56 @Bean new *
57 public Binding deadLetterBinding(Queue deadLetterQueue, DirectExchange deadLetterExchange) {
58     return BindingBuilder.bind(deadLetterQueue).to(deadLetterExchange).with( routingKey: "notification.dead");
59 }
60 }
```

145. Dead Letter Queue mensajes fallidos



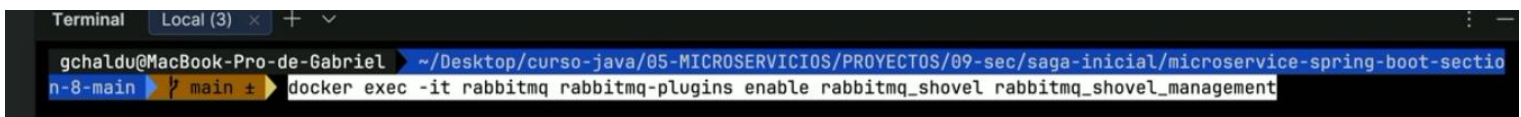
```
@Configuration @ydochoadev *
public class RabbitMQConfig {
    /* Bean para convertir la serialización java a Jackson
    para convertir los mensajes a json */
    @Bean @ydochoadev
    public MessageConverter messageConverter() { return new JacksonJsonMessageConverter(); }

    // Cola: Donde se almacenan los mensajes esperando ser leídos
    @Bean(name = "notificationQueueBean") @ydochoadev *
    public Queue notificationQueue() {
        // duration: SI RabbitMQ se reinicia, la cola sobrevive
        return QueueBuilder.durable( name: "notification-queue")
            .withArgument("x-dead-letter-exchange", "notification-dlx")
            .withArgument("x-dead-letter-routing-key", "notification.dead")
            .build();
    }
}
```

Para poder reenviar las notificaciones, se debe habilitar un plugin, y eso se realiza en el contenedor de docker:

Ejecutar:

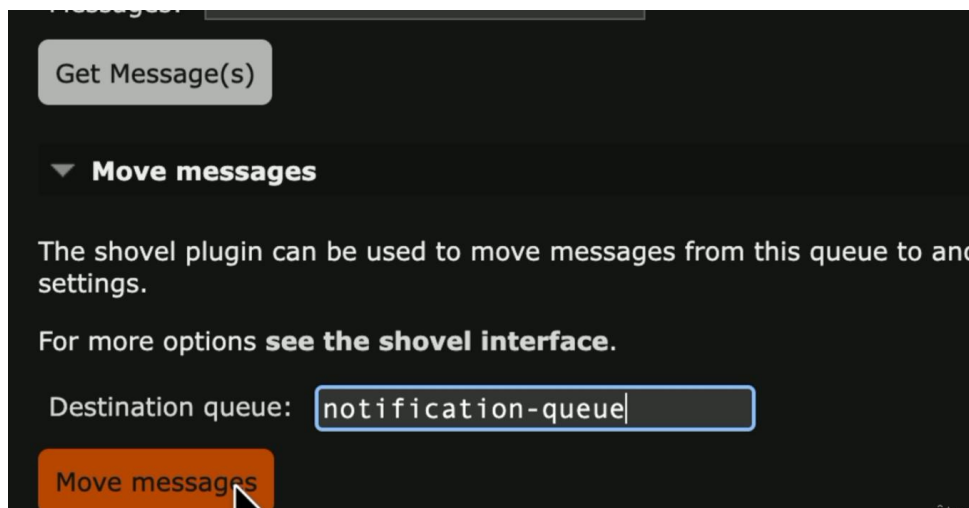
\$> docker exec -it rabbitmq rabbitmq-plugins enable rabbitmq_shovel rabbitmq_shovel_management



```
gchaldu@MacBook-Pro-de-Gabriel ~/Desktop/curso-java/05-MICROSERVICIOS/PROYECTOS/09-sec/saga-inicial/microservice-spring-boot-sectio
n-8-main ➜ main ± docker exec -it rabbitmq rabbitmq-plugins enable rabbitmq_shovel rabbitmq_shovel_management
```

<https://gist.github.com/gchaldu/c11615ec71a526ae513ae1cf62914f9a>

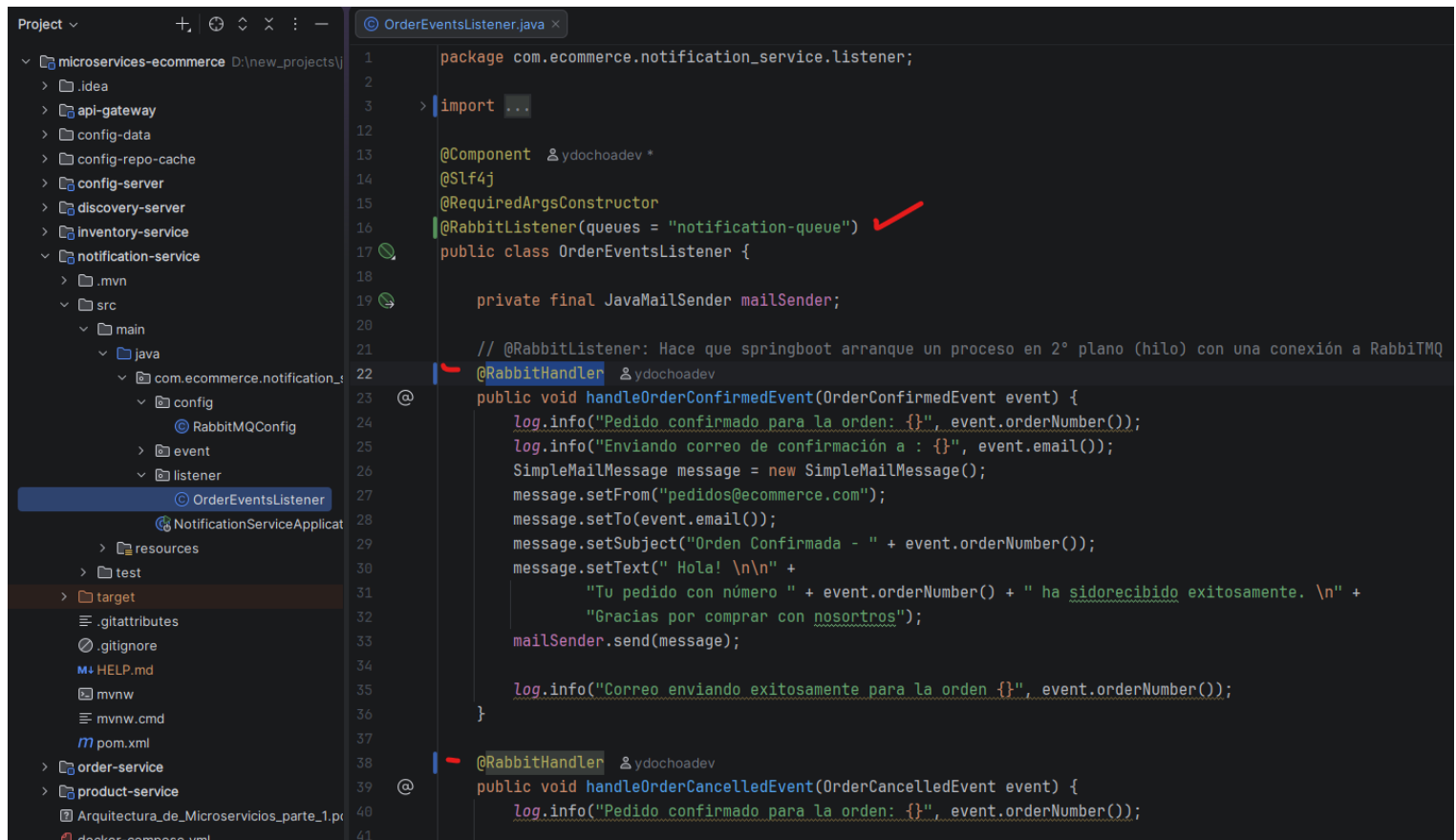
En la consola RabbitMQ:



146. Patrón Saga: método 2 - @RabbitHandler

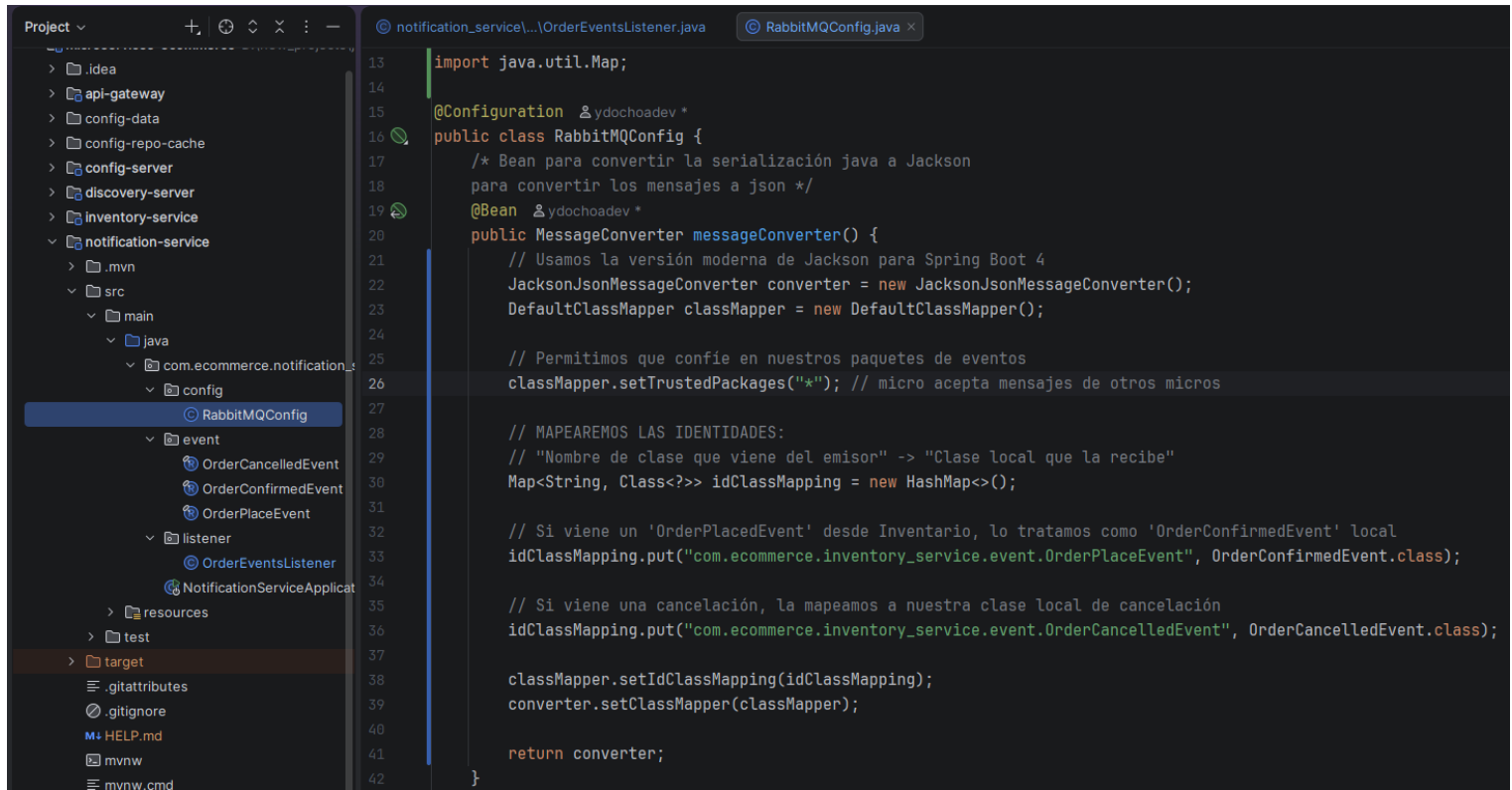
→ Competencia de consumidores, se tienen varios RabbitMQ escuchando la misma cola, esto crea múltiples hilos compitiendo por el mismo mensaje.

→ Cambiar el decorador @RabbitListener(queues = "notification-queue") a nivel de clase y no de método (los métodos tendrán @RabbitHandler):



➔ Luego, en la configuración de RabbitMQ de notification-service, se debe cambiar el converter:

** Antes de que el mensaje llegue al listener, el Bean hará lo siguiente:



```
13 import java.util.Map;
14
15 @Configuration &ydochoadev *
16 public class RabbitMQConfig {
17     /* Bean para convertir la serialización java a Jackson
18     para convertir los mensajes a json */
19     @Bean &ydochoadev *
20     public MessageConverter messageConverter() {
21         // Usamos la versión moderna de Jackson para Spring Boot 4
22         JacksonJsonMessageConverter converter = new JacksonJsonMessageConverter();
23         DefaultClassMapper classMapper = new DefaultClassMapper();
24
25         // Permitimos que confíe en nuestros paquetes de eventos
26         classMapper.setTrustedPackages("*"); // micro acepta mensajes de otros micros
27
28         // MAPEAREMOS LAS IDENTIDADES:
29         // "Nombre de clase que viene del emisor" -> "Clase local que la recibe"
30         Map<String, Class<?>> idClassMapping = new HashMap<>();
31
32         // Si viene un 'OrderPlacedEvent' desde Inventario, lo tratamos como 'OrderConfirmedEvent' local
33         idClassMapping.put("com.ecommerce.inventory_service.event.OrderPlaceEvent", OrderConfirmedEvent.class);
34
35         // Si viene una cancelación, la mapeamos a nuestra clase local de cancelación
36         idClassMapping.put("com.ecommerce.inventory_service.event.OrderCancelledEvent", OrderCancelledEvent.class);
37
38         classMapper.setIdClassMapping(idClassMapping);
39         converter.setClassMapper(classMapper);
40
41         return converter;
42     }
```